

Universidad Tecnológica de La Habana

“José Antonio Echeverría”



Jornada Científica Estudiantil a nivel de universidad

**Nueva versión de la Biblioteca de Heurísticas de
Construcción para Problemas de Planificación de Rutas
de Vehículos**

Autor: Ananda de la Caridad Morales Morales

Carrera: Ingeniería Informática

Año académico: 5to

Tutores:

Profesor Auxiliar Dr. C. Isis Torres Pérez [0000-0002-1079-8285]

itorres@ceis.cujae.edu.cu

Profesor Titular Dr. C. Alejandro Rosete Suárez [0000-0002-4579-3556]

rosete@ceis.cujae.edu.cu

La Habana, 2024

“Año 66 de la Revolución”

Planilla de localización

Autor:

Ananda de la Caridad Morales Morales [0009-0003-9208-8029] *

amorales@ceis.cujae.edu.cu

nanda.snow.black@gmail.com

Tutores:

Dr. C. Isis Torres Pérez [0000-0002-1079-8285] *

itorres@ceis.cujae.edu.cu

tp.isis@gmail.com

Dr. C. Alejandro Rosete Suárez [0000-0002-4579-3556] *

rosete@ceis.cujae.edu.cu

* Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE), Facultad de Ingeniería Informática, La Habana, Cuba.

Resumen

Dentro de los tipos de problemas de optimización se encuentran los problemas de optimización combinatoria, que incluye a los Problemas de Planificación de Rutas de Vehículos. Estos problemas cuentan con una flota de vehículos, buscan satisfacer las demandas de clientes dispersos geográficamente y tienen como objetivo minimizar costos siempre que se cumplan con las restricciones existentes. A diario se producen situaciones prácticas en el mundo, que conllevan al surgimiento de nuevas variantes. Una alternativa para resolver estos problemas, son las heurísticas de construcción, cuyo objetivo es encontrar una primera solución cercana al óptimo. Sin embargo, la mayoría de las heurísticas se limitan a la variante básica, y son pocas las bibliotecas que implementan dichos métodos. En este contexto surge la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos, conocida como BHCVRP, con el propósito de dar solución a cuatro tipos de problemas de planificación de rutas de vehículos mediante el empleo de siete heurísticas de construcción. En este trabajo se presenta una nueva versión de dicha biblioteca implementada en los lenguajes de programación Java y Python, que incorpora dos nuevas heurísticas de construcción: Inserción de Kilby y el Algoritmo de Ahorro basado en *Matching*. Incluye tratamiento de excepciones propias y el empleo del patrón de diseño *Template*. Por último, se comparan los resultados alcanzados con la versión anterior de BHCVRP para verificar el correcto funcionamiento de las nuevas versiones; así como con la herramienta OR-Tools para comparar la calidad de las soluciones obtenidas.

Palabras claves: BHCVRP, diseño de software, heurísticas de construcción, patrones de diseño, Python, VRP.

Índice

Introducción.....	3
Declaración de objetivos	7
Control semántico	8
1. Fundamentos teóricos.....	10
1.1 Bibliotecas que implementan métodos heurísticos	11
1.2 Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos.....	14
2. Diseño de la solución	16
3. Análisis experimental	20
3.1 Configuración de los experimentos.....	20
3.2 Análisis de los resultados.....	22
Conclusiones.....	26
Recomendaciones.....	28
Referencias bibliográficas	29
Anexos	35

Introducción

Un problema de optimización combinatoria tiene como objetivo encontrar la mejor solución entre un conjunto finito de posibilidades. Cada solución está formada por una combinación de elementos de un conjunto dado. Estos problemas suelen involucrar la selección de un subconjunto de elementos que maximice o minimice una función objetivo, sujeto a ciertas restricciones (Papadimitriou & Steiglitz, 1998). En esta categoría se encuentra un problema bastante frecuente en los sistemas logísticos, que es la distribución de bienes y servicios desde ciertos depósitos a usuarios dispersos geográficamente. En la mayoría de las ocasiones, se desea mejorar la planificación eficiente de las rutas de los vehículos, determinando cómo visitar a los clientes para satisfacer sus demandas (Toth & Vigo, 2002). En este contexto, surge el denominado Problema de Planificación de Rutas de Vehículos (VRP, por las siglas en inglés de *Vehicle Routing Problem*) (Eksioglu, Vural, & Reisman, 2009).

De manera general, un VRP busca diseñar un conjunto de rutas para que una flota de vehículos preste servicio a un conjunto de clientes, sujeto a una serie de restricciones. Este problema es empleado en la gestión de la cadena de suministro en la entrega física de bienes y servicios. Existen diversas variantes de VRP y se formulan en función de la naturaleza de los bienes transportados, la calidad del servicio requerido, las características de los clientes y los vehículos, entre otros (Kumar & Panneerselvam, 2012). Según (Dantzig & Ramser, 1959), los problemas de planificación de rutas de vehículos se introducen en 1959 y se describe como una variante generalizada del Problema del Viajante de Comercio (TSP, por las siglas de *Traveling Salesman Problem*) (Applegate, 2006). En 1964, se propone el primer algoritmo efectivo para resolver VRP, denominado Algoritmo de Ahorros (Clarke & Wright, 1964). A partir de estos sucesos, incrementan las investigaciones y trabajos en el área de la planificación de rutas de vehículos. Por consiguiente, surgen dos enfoques: definición de modelos que incorporen cada vez más características de la vida real y búsqueda de algoritmos para resolver eficientemente estos problemas (Zirour, 2008).

Para resolver situaciones prácticas se proponen modelos básicos que pueden ser extendidos a diferentes campos de la logística y el transporte (Konstantakopoulos,

Gayialis, & Kechagias, 2022; Laporte, 1992). Algunos ejemplos de aplicaciones prácticas de VRP se encuentran en el transporte obrero, las rutas de ómnibus, la recogida de desechos sólidos, la distribución del periódico y correo, entre otros. Estas situaciones cotidianas pueden ser modeladas con sus características, a través de las diferentes variantes de VRP existentes. Dentro de las variantes más conocidas se encuentran: el Problema de Planificación de Rutas de Vehículos con Capacidades (CVRP) (Arifita & Rakhmawati, 2023), el Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (MDVRP) (Morsidi & Panessai, 2023), el Problema de Planificación de Rutas de Vehículos con Flota Heterogénea (HFVRP) (Máximo, Cordeau, & Nascimento, 2022), el Problema de Planificación de Rutas de Camiones y Remolques (TTRP) (Accorsi & Vigo, 2020), el Problema de Planificación de Rutas de Autobuses Escolares (SBRP) (Rita M. Newton & Thomas, 1969), entre otros (Golden, Wang, & Wasil, 2023).

Los problemas de planificación de rutas de vehículos son mayormente complejos de resolver para instancias que contengan un número de clientes elevado. Este fenómeno se debe al esfuerzo computacional requerido y a que los VRP pertenecen a la clase de problemas NP-duro (Laporte, 1992). Para desempeñar esta tarea, en ocasiones se necesita comprometer requisitos como la optimalidad y construir una buena solución, a pesar de no garantizar el óptimo. Para eso, se aplican los algoritmos heurísticos, como las heurísticas y metaheurísticas (Talbi, 2009).

Particularmente, el propósito de las heurísticas es guiar el proceso de búsqueda en la mejor dirección y sugerir qué camino tomar cuando existen varias opciones disponibles (Fisher & Jaikumar, 1981; Romanycia & Pelletier, 1985). El término heurístico se refiere a un procedimiento que trata de aportar soluciones a un problema con un buen rendimiento, en cuanto a calidad de soluciones y recursos empleados. Esto significa que constituyen métodos más sencillos que brindan soluciones satisfactorias a un problema dado mediante algoritmos específicos. Además, pueden ofrecer la solución final del problema o ser aplicadas dentro del contexto de las metaheurísticas (Romanycia & Pelletier, 1985; Zirour, 2008).

Por otra parte, las heurísticas se clasifican en dos grupos: heurísticas de construcción y heurísticas de mejora (Clancey, 1985; Martí, 2003). Las heurísticas de construcción se

encargan de construir literalmente una solución del problema, paso por paso (Martí, 2003). Sin embargo, las heurísticas de mejora parten de una solución y reparan las rutas previamente construidas, con el objetivo de conseguir una determinada mejoría (Romanycia & Pelletier, 1985). Dentro de las heurísticas de construcción clásicas de la literatura, utilizadas mayormente para la resolución de los VRP, se encuentran: el Algoritmo de Ahorros (Clarke & Wright, 1964), el Algoritmo de Barrido (Nurcahyo, Alias, Shamsuddin, & Sap, 2002), la Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (Medina, La Rota, & Castro, 2011), Heurística de Inserción Secuencial de Mole & Jameson (Mole & Jameson, 1976), entre otras (Olivera, 2004).

Debido a la necesidad de resolver problemas de optimización combinatoria utilizando dichos algoritmos y favorecer la reutilización de sus códigos, es apropiado agruparlos dentro de componentes de software. Actualmente, existen pocas bibliotecas que implementan métodos heurísticos para resolver, ya sea VRP u otros problemas de optimización. Algunos de los componentes de software que cumplen con dichas características son:

- BiCIAM: biblioteca de clases en Java, que implementa un modelo unificado de metaheurísticas mono-objetivo y multi-objetivo (Pérez-Pérez, Sánchez-Ansola, & Rosete-Suárez, 2021).
- VRPH: biblioteca de heurísticas de búsqueda local para VRP en C++ (Groër, Golden, & Wasil, 2010).
- BHCVRP: biblioteca de heurísticas de construcción para VRP en Java (Díaz-Subi, 2016).
- OR-Tools: software de código abierto desarrollado principalmente en Python, para resolver problemas de optimización combinatoria, incluyendo VRP en sus distintas variantes como CVRP y HFVRP (Didier et al., 2023).
- JSprit: herramienta de código abierto desarrollada en Java, que implementa el principio *Ruin and Recreate* (Lima, 2017) y resuelve varias variantes VRP (Mahmoud, Chouaki, Hörl, & Puchinger, 2022).

- VROOM: software de código abierto desarrollado en C++, para resolver VRP utilizando heurísticas para obtener la solución inicial, en dependencia de la variante del problema (Karkula, Duda, & Skalna, 2019).

Los componentes de software antes mencionados permiten resolver VRP. De este conjunto, BHCVRP es la única que permite hacerlo a través de heurísticas de construcción exclusivamente. Generalmente, en la literatura se evidencia la carencia de bibliotecas de heurísticas para la resolución de problemas de optimización combinatoria, debido a que la mayor parte del desarrollo se centra en las metaheurísticas. Además, existen pocas adaptaciones de las heurísticas a las distintas variantes VRP, ya que no se consideran las características propias de cada uno de estos problemas. Por último, la mayoría de los componentes de software existentes para solucionar estos problemas están desarrollados para C++ y Java. Debido al crecimiento de la comunidad en Python para trabajos con Inteligencia Artificial y problemas de optimización, sería interesante profundizar en cómo solucionar VRP con heurísticas desde este lenguaje (Calderón Calderón, 2020; PAREJO et al., 2020).

La Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP) es una propuesta realizada por el proyecto de Optimización y Metaheurísticas de la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE) (Díaz-Subi, 2016). Esta biblioteca está desarrollada en Java y contiene la adaptación de siete heurísticas de construcción de la literatura para las variantes CVRP, MDVRP, HFVRP y TTRP. A pesar de constituir un aporte, presenta deficiencias en su diseño y código fuente relacionado con las heurísticas implementadas, así como dificultades en la interoperabilidad con otros componentes.

A partir de la situación descrita anteriormente se plantea como problema a resolver la existencia de deficiencias en la arquitectura de BHCVRP, teniendo en cuenta la manera en que se encuentran implementadas las heurísticas para resolver cada variante VRP.

En este trabajo, el objeto de estudio se enmarca en los problemas de optimización combinatoria, los algoritmos heurísticos y el diseño de software. Específicamente, el campo de acción se enfoca en los Problemas de Planificación de Rutas de Vehículos, las heurísticas de construcción y los patrones de diseño.

Declaración de objetivos

Para dar solución al problema planteado se identifica como objetivo general: Desarrollar una nueva versión de BHCVRP en Java y Python, que permita incorporar otras heurísticas de construcción de la literatura para diferentes variantes de problemas de planificación de rutas de vehículos. Para dar cumplimiento al objetivo general, se definen los siguientes objetivos específicos:

1. Caracterizar el marco teórico investigativo sobre los problemas de planificación de rutas de vehículos, el uso de los algoritmos heurísticos para resolverlos y los componentes de software que agrupan algoritmos heurísticos para su reutilización.
2. Rediseñar la arquitectura de la biblioteca de clases para la utilización de otras heurísticas de construcción en diferentes variantes VRP.
3. Implementar adaptaciones de otras heurísticas de construcción para los problemas de planificación de rutas de vehículos.
4. Validar el funcionamiento de las heurísticas adaptadas a través de un conjunto de experimentos con instancias de la literatura.

Este trabajo solo se enfoca en los objetivos específicos 1 y 2. Del objetivo específico 3, se estarán realizando las tareas relacionadas con las nuevas heurísticas de construcción, solo en el lenguaje de programación Python. El objetivo específico 4 será cumplido parcialmente, ya que los experimentos se harán con un alcance limitado. Con vistas a la tesis se incorporarán nuevas variantes VRP, nuevas heurísticas de construcción, otros métodos de post-optimización, entre otras tareas. Por último, los nuevos aportes abarcan implementaciones en ambos lenguajes de programación, Java y Python.

El cumplimiento del objetivo general tiene como valor práctico un aporte significativo en la resolución de diferentes VRP a través de la biblioteca desarrollada en Python. Se propone como artefacto de salida un diseño de software, según la clasificación presentada en (Offermann, Blom, Schönherr, & Bub, 2010). Es importante destacar que los resultados obtenidos ponen a disposición de la comunidad científica un componente de software que brinda nuevos algoritmos para la resolución de distintas variantes de

VRP. Además, la biblioteca permite la colaboración con otros componentes que emplean metaheurísticas o sistemas de información geográficos con servicios de planificación de rutas de vehículos.

Control semántico

A continuación, se conceptualizan los términos más complejos que no se explican en la introducción:

- **Diseño de software:** es el proceso de definir la arquitectura, componentes, interfaces y otros aspectos de un sistema de software para satisfacer requisitos específicos. Este proceso implica la creación de una solución estructurada que guíe la implementación del software. Un diseño de software como artefacto de salida en una investigación se refiere a la representación formal y documentada de cómo se planea implementar un sistema de software específico. Este artefacto es el resultado de un proceso de investigación que puede involucrar el análisis de problemas, la identificación de requisitos y la propuesta de soluciones (Capilla Sevilla, 2022).
- **Metaheurísticas:** constituyen una clase de algoritmos aproximados diseñados para resolver problemas difíciles de optimización combinatoria. Dichos métodos son procedimientos iterativos que combinan de manera inteligente diferentes conceptos para explorar el espacio de búsqueda y guían una heurística subordinada (Martí, 2003). Además, las metaheurísticas son aplicadas cuando no existe un método exacto de resolución a un problema de optimización, requiere demasiado costo computacional o no se conoce la solución óptima. Por último, las metaheurísticas permiten enfrentar problemas de gran tamaño mediante la búsqueda de buenas soluciones en un tiempo razonable (Talbi, 2009).
- **Patrones de diseño:** permiten identificar, en el contexto de la programación orientada a objetos, problemas concretos junto con sus soluciones. Constituyen una forma de documentar la experiencia acumulada en las estrategias empleadas para resolver problemas específicos dentro de un contexto determinado (Eckel, 2003). Según los patrones GoF (Gamma, Helm, Johnson, & Vlissides, 1995), pueden clasificarse en estructurales, creacionales o de comportamiento.

- **Patrón de diseño *Template Method*:** es un patrón estructural que define el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclases. La intención consiste en permitir que las subclases redefinan ciertos pasos de un algoritmo sin alterar la estructura del mismo. Además, facilita la reutilización de código y mantiene la flexibilidad en la implementación de algoritmos (Gamma et al., 1995). Una de sus principales ventajas es que permite cambiar la secuencia de pasos de un algoritmo sin modificar su estructura. Esto es especialmente útil cuando diferentes variantes de un algoritmo comparten la misma estructura, pero difieren en sus pasos. Al definir estos pasos en métodos individuales, las subclases pueden sobrescribir esos métodos para personalizar el comportamiento del algoritmo según sea necesario. Esto mejora la cohesión y reduce la duplicación de código, ya que la lógica común se encapsula en la clase base (Gamma et al., 1995).

- **Problemas NP-duros:** constituyen problemas que actualmente no cuentan con un algoritmo capaz de obtener el estado óptimo en un tiempo polinomial, pero que tampoco se ha probado que no pueda existir dicho algoritmo, para al menos un problema. Además, no han podido ser reducidos a problemas que pudieran ser resueltos por un algoritmo no determinístico en un tiempo polinomial. Por tanto, son considerados intratables o muy difíciles de resolver (Hochba, 1997; Laporte, 1992).

- **Problema del Viajante de Comercio (TSP):** constituye el punto de partida para la formulación de otros problemas de optimización combinatoria más complejos y utilizados en situaciones prácticas, como es el caso de los problemas de planificación de rutas de vehículos. El TSP se basa en un conjunto de n ciudades y el costo C_{ij} que se obtiene al viajar de un punto a otro. Posee como objetivo fundamental encontrar la ruta de costo mínimo para visitar todas las ciudades pasando solo una vez por cada una de ellas, y regresando al punto de partida. Por tanto, el TSP es considerado el problema de planificación de rutas de vehículos más simple (Applegate, 2006).

1. Fundamentos teóricos

Los modelos básicos creados para los VRP son utilizados para resolver situaciones prácticas en el campo de la logística y el transporte (Cordeau & décisions, 2000; Laporte, 1992). Dentro de las aplicaciones prácticas del VRP se encuentran: el transporte obrero, las rutas de ómnibus, la recolección de desechos sólidos, la distribución del periódico y correo, entre otros. Dichas situaciones cotidianas, pueden ser modeladas a través de diferentes variantes VRP existentes o la fusión de estas, respetando sus características. En (Golden et al., 2023) se realiza una investigación acerca de la evolución de los VRP desde 2005 hasta 2022, teniendo en cuenta las distintas variantes y nuevas características que surgen en problemas de la vida real.

Al abordar este tipo de problemas, es común utilizar algoritmos heurísticos para encontrar soluciones. Las heurísticas se utilizan ampliamente como alternativa para garantizar los mejores resultados posibles en los VRP. La mayoría de las heurísticas poseen diversas naturalezas y son diseñadas para problemas específicos, dificultando su generalización o aplicación a problemas similares. Sin embargo, dentro de los casos especiales se encuentran los métodos de construcción, con el objetivo de construir literalmente paso a paso una solución del problema (Martí, 2003). Las heurísticas de construcción (HC) para problemas de planificación de rutas de vehículos se dividen en cuatro grupos según (Daza, Montoya, & Narducci, 2009):

- Basadas en ahorros (*Saving Based*): mediante ahorros producidos por la sustitución de nodos en la ruta por otros que aún no están, se mantiene un proceso de constante intercambio mientras se cumplan las restricciones. Dentro de los algoritmos más conocidos se encuentran el Algoritmo de Ahorros (*Save Algorithm*) (Clarke & Wright, 1964) con su versión secuencial y paralela, así como la variante basada en *Matching* (Olivera, 2004).
- De inserción o basadas en coste (*Cost Based*): son empleadas para generar un conjunto de rutas iniciales. Se construye una solución mediante constantes inserciones de nodos (Mole & Jameson, 1976). Dentro de las heurísticas más empleadas se encuentran Vecino más cercano, Inserción Secuencial de Mole & Jameson (Mole & Jameson, 1976) e Inserción de Kilby (Algarra & Pelta, 2012).

- Ruta primero – Asignar después (*Route first – Cluster second*): se determina el recorrido para visitar a todos los clientes y luego se particiona en varias rutas para que cada una sea factible, garantizando de esta manera el cumplimiento de las restricciones (Olivera, 2004). Dentro de los algoritmos que poseen este comportamiento se encuentran los Algoritmos de Pétalos (González Restrepo & Gómez Veloza, 2019).
- Asignar primero – Ruta después (*Cluster first – Route second*): se busca generar un grupo de clientes que pertenezcan a una misma ruta en la solución final y luego se crea una ruta para cada grupo que visite a todos sus clientes determinando el orden en que serán visitados (Olivera, 2004). Dentro de los algoritmos más conocidos se encuentra el Algoritmo de Barrido (Nurcahyo et al., 2002), el Algoritmo de Gillet-Miller (Gillett & Miller, 1974) y la Heurística de Asignación Generalizada de Fisher y Jaikumar (Fisher & Jaikumar, 1981).

La mayoría de las heurísticas de construcción están diseñadas para el VRP más sencillo, es decir, el problema de planificación de rutas de vehículos con capacidades (CVRP). Por tanto, deben realizarse adaptaciones a estos métodos constructivos para aplicarlos en otras variantes.

1.1 Bibliotecas que implementan métodos heurísticos

Para solucionar problemas de optimización resulta importante el uso de una biblioteca que implemente métodos heurísticos que garanticen la reutilización de los mismos. La mayor parte de las investigaciones se han centrado en el desarrollo de bibliotecas con algoritmos metaheurísticos, ya que son aplicables de manera general a cualquier tipo de problema de optimización. Dentro de las principales bibliotecas en la actualidad que implementan algoritmos metaheurísticos, se encuentran: BiCIAM (Pérez-Pérez et al., 2021), METSlib (Maischberger, 2011), MALLBA (Enrique Alba et al., 2002), MOMHLib++ (Florios & Mavrotas, 2014), *Open Metaheuristics* (Díaz-Subi, 2016), *Operation Research Tools* (OR-Tools) (Didier et al., 2023), Jsprit (Mahmoud et al., 2022) y OptaPlanner (Souza, 2020).

Sin embargo, para la resolución de problemas de planificación de rutas de vehículos e incluso otros problemas de optimización, existen escasas bibliotecas que implementen algoritmos heurísticos. A continuación, se mencionan las bibliotecas encontradas en la literatura para este fin: Biblioteca de heurísticas de búsqueda local para el problema de ruteo de vehículos (VRPH) (Baldacci, Toth, & Vigo, 2010), Biblioteca de código abierto para búsqueda heurística (Hipster) (Díaz-Subi, 2016), *Vehicle Routing Open-source Optimization Machine* (VROOM) (Karkula et al., 2019) y Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP) (Díaz-Subi, 2016).

A continuación, se realiza una comparación de las bibliotecas mencionadas anteriormente, teniendo en cuenta un conjunto de características. En la Tabla 1, se muestran los criterios establecidos para dicho estudio. Teniendo en cuenta el análisis comparativo realizado, se puede arribar a las siguientes conclusiones:

- Las bibliotecas Hipster y MALLBA resuelven problemas de optimización diferentes a los VRP que son los de interés en esta investigación.
- Los componentes de software VRPH, BHCVRP, VROOM, OR-Tools y Jsprit poseen en común la característica de solucionar la variante CVRP. De manera general, entre estas se solucionan los casos de MDVRP, HFVRP y VRPTW.
- La mayoría de las bibliotecas de clases implementan metaheurísticas.
- La única biblioteca interoperable es OR-Tools, aunque existe un servicio web de BHCVRP que se encuentra en desuso (Díaz-Subi, 2016).
- Todas las bibliotecas de metaheurísticas son extensibles, mientras que las bibliotecas de heurísticas solo cuentan con Hipster y BHCVRP que extienden de manera parcial. Por tanto, se pueden agregar nuevos algoritmos de manera sencilla.
- De todas las bibliotecas solo BiCIAM, Hipster, OR-Tools y BHCVRP son libres.
- Solo las bibliotecas de clases BHCVRP y VROOM implementan heurísticas de construcción para la resolución de variantes VRP.

Tabla 1: Comparación de bibliotecas de clases que implementan métodos heurísticos.

Bibliotecas	Características						
	Resuelve VRPs	Implementa HC	Implementa MH	Lenguaje de programación	Interoperable	Extensible	Libre y de código abierto
BiCIAM	✓		✓	Java		✓	✓
METSlib	✓		✓	C++		✓	
MALLBA		✓		C++			
MOMHLib++	✓		✓	C++		✓	
<i>Open Metaheuristics</i>	✓		✓	C/C++		✓	
OR-Tools	✓		✓	C++/C#/Java/Python	✓	✓	✓
Jsprit	✓		✓	Java		✓	
OptaPlanner	✓		✓	Java		✓	
VRPH	✓	✓		C/C++			
Hipster				Java		✓	✓
VROOM	✓	✓	✓	C++		✓	
BHCVRP	✓	✓		Java	✓	✓	✓

Por último, resulta interesante analizar las tecnologías empleadas en el desarrollo de estos componentes de softwares que solucionan problemas de planificación de rutas de vehículos. A partir del análisis comparativo plasmado anteriormente, se evidencia que la mayoría de estos componentes se han desarrollado en los lenguajes de programación C++ y Java. Sin embargo, en los últimos tiempos la comunidad de Python ha crecido exponencialmente, sobre todo en trabajos relacionados con Inteligencia Artificial y problemas de optimización (Calderón Calderón, 2020; PAREJO et al., 2020). A pesar de la existencia de OR-Tools en este lenguaje, se identifican deficiencias; por ejemplo, se aplican pocas heurísticas de construcción para solucionar VRP y las heurísticas existentes son deterministas, lo cual limita el espacio de búsqueda de nuevas soluciones.

1.2 Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos

BHCVRP es una biblioteca de clases que implementa heurísticas de construcción para resolver problemas de planificación de rutas de vehículos. Esta biblioteca fue desarrollada en (Díaz-Subi, 2016), en el lenguaje de programación Java y contiene la adaptación de siete heurísticas clásicas para las variantes CVRP, MDVRP, HFVRP y TTRP. Las heurísticas implementadas son: el Algoritmo de Barrido, las dos versiones del Algoritmo de Ahorros (secuencial y paralela), la Heurística del Vecino más Cercano con Lista de Candidatos Restringidos, la Heurística de Inserción de Mole & Jameson, la Heurística de Christofides, Mingozzi y Toth, y un método aleatorio (Díaz-Subi, 2016).

El enfoque utilizado en la estructuración en capas de BHCVRP está basado en reutilización (Díaz-Subi, 2016), y se ilustra a continuación, en la Figura 1:

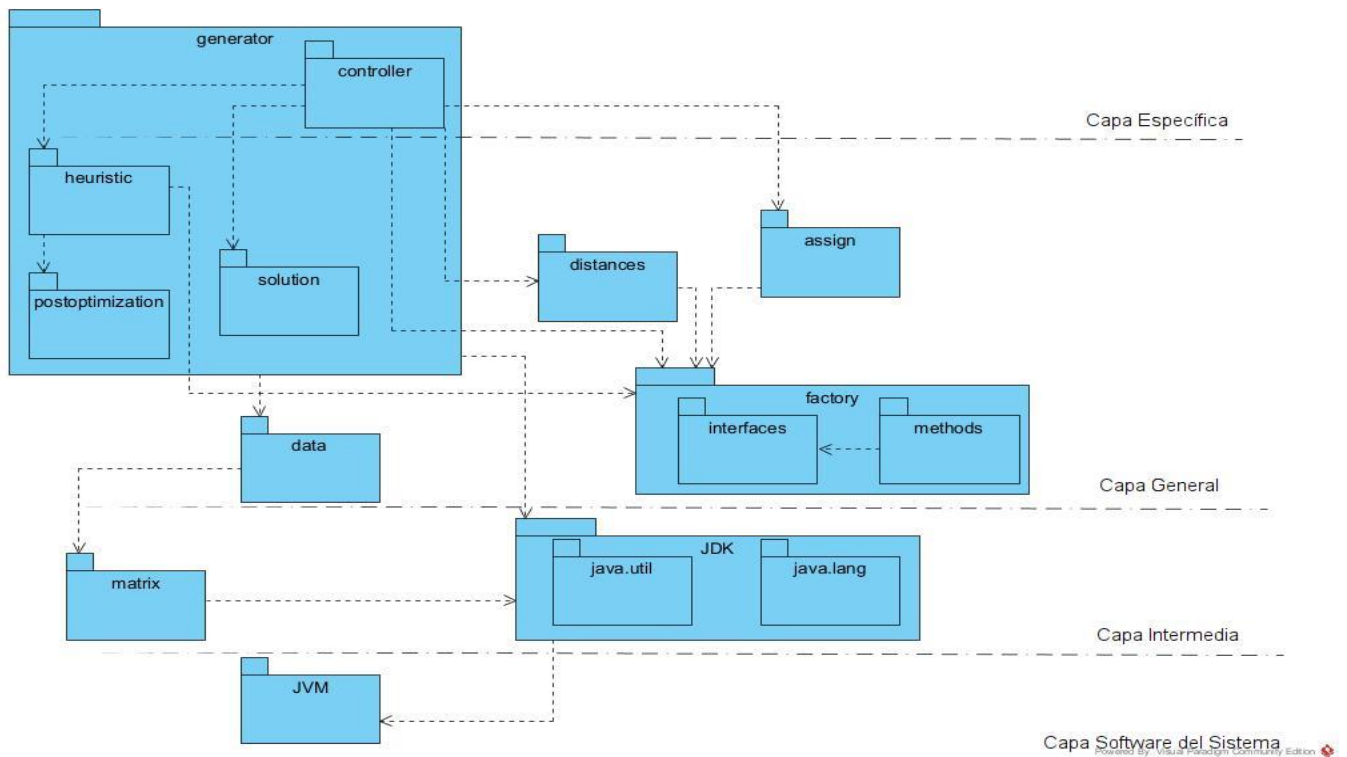


Figura 1: Patrón n-capas con enfoque basado en reutilización de BHCVRP (Díaz-Subi, 2016).

A partir del estudio y el análisis de la arquitectura y del código fuente de la biblioteca de clases BHCVRP, se han identificado un conjunto de deficiencias que no permiten realizar un uso eficiente de la misma. A continuación, se describen algunas de estas deficiencias y posibles mejoras:

- No presenta un paquete para tratar excepciones propias relacionadas con parámetros como la distancia, tiempo, capacidad, existencia o no de un depósito, cliente, vehículo, entre otros. Solamente se utilizan las que brinda el paquete *java.lang* propio del lenguaje de programación.
- El paquete *assign*, debe actualizarse por el nuevo componente BHAVRP (Pérez, 2022) donde se identifica en la variante MDVRP posibles mejoras en la incorporación de nuevos métodos de asignación, así como admitir por parte del usuario una asignación predeterminada.

- Además, la biblioteca debe ser capaz de adaptar nuevas heurísticas de construcción de la literatura como Inserción de Kilby (Algarra & Pelta, 2012) y Algoritmo de Ahorros basado en *Matching* (Olivera, 2004), así como nuevas variantes VRP como (Pérez-Pérez et al., 2021; Piqueras, 2002; Tangarife Álvarez, 2017).
- Como parte de las buenas prácticas que se siguen en el diseño de software, se necesitan aplicar patrones y principios de diseño para favorecer la flexibilidad y reutilización, ya que solo se aplican *Singleton* y *Factory Method* (Sommerville & Wesley, 2011).
- Existe una deficiencia arquitectónica, ya que la biblioteca no brinda servicio diferenciado para dos tipos de usuarios: uno experto con conocimientos de optimización con necesidades de configurar la mayor cantidad de parámetros posibles y otro usuario con conocimientos de su problema VRP real con una configuración predeterminada a partir de los mejores resultados alcanzados en experimentos. Actualmente, no se proporciona una configuración por defecto.
- A la biblioteca le falta interoperabilidad, lo cual implica que no se comunique e integre con otros componentes o aplicaciones, intercambie datos, se utilice en diferentes entornos, siga con estándares de codificación, de la manera más efectiva posible. En primera instancia, sólo se pensó ser capaz de funcionar con la herramienta TransO (Nerey, 2017) y el componente BHAVRP (Machado, 2019; Pérez, 2022). Aunque existe una versión web en desarrollo (Díaz-Subi, 2016).

2. Diseño de la solución

A partir del análisis realizado en la sección 1.2 y como resultado de las modificaciones realizadas a la biblioteca, se muestran en la Figura 2, el diseño arquitectónico de la nueva versión de BHCVRP para los lenguajes de programación Java y Python. Se utilizan colores diferentes para reflejar los cambios introducidos. En el caso de los paquetes que son nuevos en la arquitectura se emplea el color verde, el color morado para aquellos paquetes que sufrieron algún tipo de cambio y el color azul para los que se mantienen. En primera instancia, la arquitectura de BHCVRP se basa en el patrón n- capas con un

enfoque basado en reutilización (Reynoso, 2004). A continuación, en la Figura 2, se muestra el diagrama de estructuración en capas para la nueva versión.

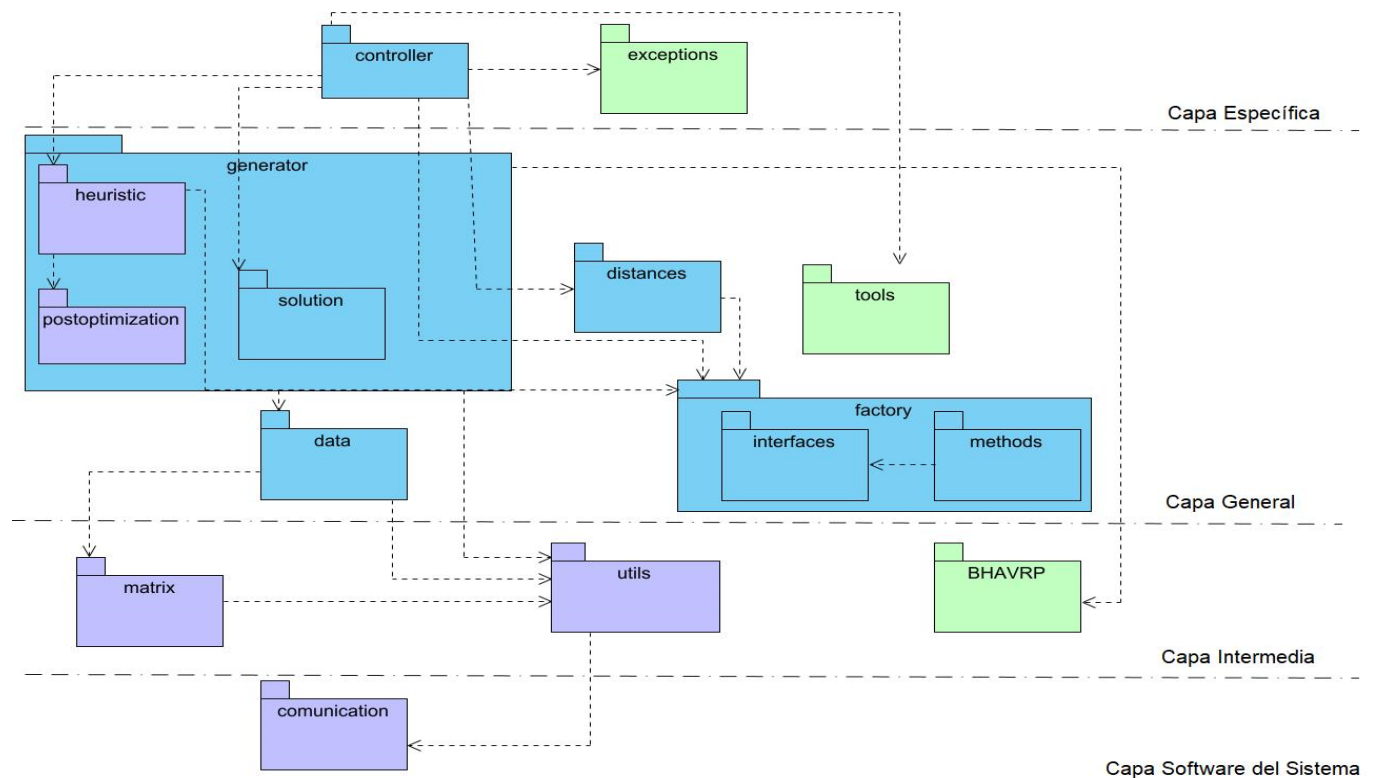


Figura 2: Patrón n-capas con enfoque basado en reutilización de la nueva versión de BHCVRP.

A continuación, en la Tabla 2, se explican las funciones de los paquetes que contiene cada capa para las versiones implementadas en Java y Python.

Tabla 2: Particularidades de cada paquete según el lenguaje de programación.

Paquetes comunes	Versión Java	Versión Python
<i>Matrix</i>	Librería <i>libmatrix</i> que contiene las clases necesarias para modelar la matriz de costo.	Librería <i>numpy</i> que contiene los métodos necesarios para el trabajo con matrices. En este caso, se aplicaría para la construcción de matrices de costo.

<i>Utils</i>	Contienen clases, implementaciones y excepciones necesarias para el desarrollo del sistema a través de las librerías <i>java.util</i> y <i>java.lang</i> .	Contienen clases, implementaciones y excepciones necesarias para el desarrollo del sistema a través de las librerías <i>typing</i> , <i>time</i> , <i>tqdm</i> y <i>sys</i> .
<i>BHAVRP</i>	BHAVRP (Pérez, 2022), con el objetivo es manejar los métodos de asignación de clientes a depósitos.	BHAVRP (Pérez, 2022) desarrollada en el lenguaje Java, a través de la librería <i>jpype</i> existente en Python.
<i>Communication</i>	Se realiza principalmente a través de llamadas de métodos y paso de objetos dentro del mismo proceso JVM.	Protocolo de comunicación <i>buffer</i> . Este consiste en tener capacidad para manejar grandes volúmenes de datos y su flexibilidad de uso.

La capa general está compuesta por paquetes necesarios para el funcionamiento de las heurísticas y la modelación de los datos del problema de planificación de rutas de vehículos que se quiere resolver. A continuación, se describen los cambios fundamentales incorporados como parte de la nueva propuesta de solución. El resto serán cambios relacionados con el trabajo de diploma.

El paquete *heuristic* se encarga de presentar todas las heurísticas disponibles para la construcción de rutas. A continuación, en la Figura 3, se muestra su diagrama de clases correspondiente. En color azul se muestran las clases que permanecen intactas, en color morado las que sufren modificaciones producto del patrón de diseño *Template Method* y en color verde las heurísticas nuevas incorporadas. Estas nuevas heurísticas fueron implementadas a partir del pseudocódigo proporcionado por (Olivera, 2004).

A continuación, se describen los métodos relacionados con la implementación del *Template Method*. La función *comun_initialize()* agrupa todos los parámetros que se repiten, mientras que *initialize_specifics()* permite a cada clase definir que más necesita. Los métodos *creating()*, *processing()* y *execute()* permiten crear las rutas, procesarlas de manera distinta según los intereses de cada heurística en particular y ejecutarlas para

obtener una solución respectivamente. Por último, el método `get_solution_initial()` obtiene la solución esperada y `template_method()` define el esquema general de trabajo de todas las heurísticas.

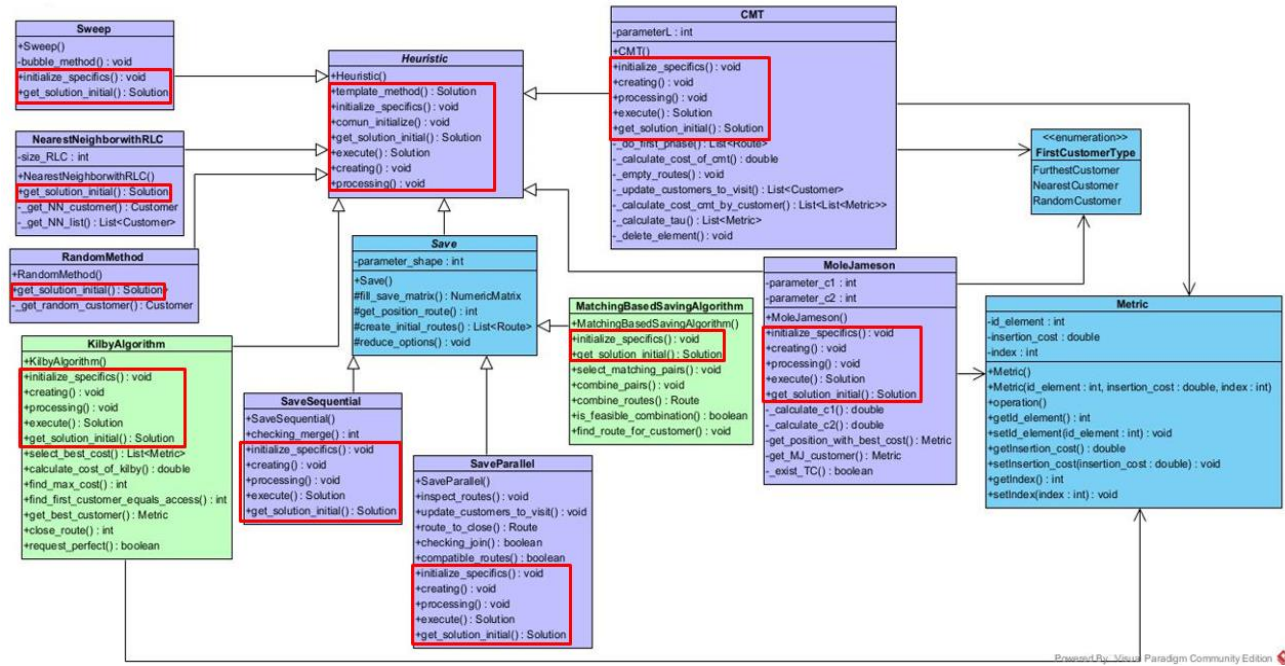


Figura 3: Diagrama de clases del paquete *heuristic* con patrón *Template Method*.

El paquete *exceptions* se encarga de manejar excepciones propias del contexto VRP, tales como capacidad, distancia, tiempo, costo, parámetros de alguna heurística, entre otros. A continuación, en la Figura 4 se muestra el diagrama de clases que representa este paquete. Se emplea el color verde para resaltar que es un paquete nuevo en su totalidad.

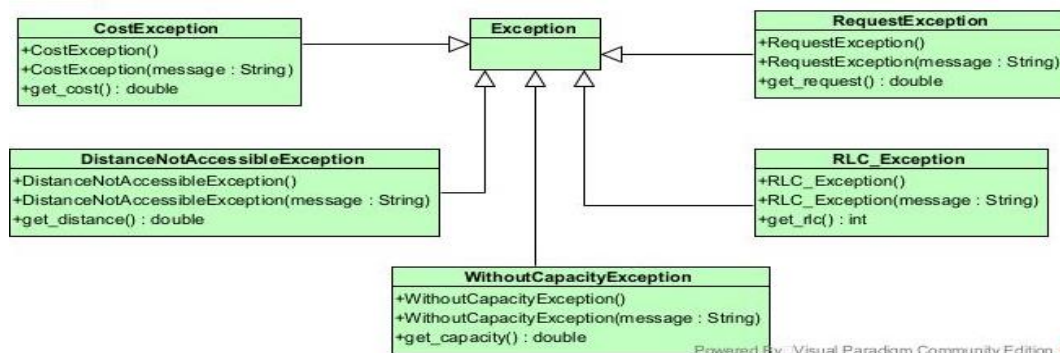


Figura 4: Diagrama de clases del paquete *exceptions*.

3. Análisis experimental

Para validar el funcionamiento de la nueva versión de BHCVRP, se decide realizar los siguientes experimentos:

- Experimento 1: consiste en ejecutar las nueve heurísticas implementadas en Python, para analizar su funcionamiento en las cuatro variantes VRP.
- Experimento 2: se realiza una comparación de los resultados actuales obtenidos en Python con los resultados de la versión anterior de la biblioteca en Java, teniendo en cuenta siete heurísticas.
- Experimento 3: se realiza una comparación de los mejores resultados obtenidos en BHCVRP con los resultados de OR-Tools para las variantes CVRP y HFVRP.

Para la ejecución de los experimentos se seleccionan para las cuatro variantes de VRP instancias de la literatura. En la sección Anexos, las Tabla 4, Tabla 5, Tabla 6 y Tabla 7, muestran las instancias empleadas en las variantes CVRP, HFVRP, MDVRP y TTRP respectivamente. Las instancias fueron seleccionadas de (E Alba, 2014) y (Chao, 2002). En el caso de las variantes CVRP y HFVRP, se realizan ligeras modificaciones.

3.1 Configuración de los experimentos

Por otra parte, todas las heurísticas adaptadas a las distintas variantes fueron implementadas en el lenguaje de programación Python versión 3.12.0, utilizando el IDE de programación PyCharm 2024.1.3 (*Community Edition*) (PyCharm, 2024). Para el caso de la versión anterior en Java (Díaz-Subi, 2016), se utilizó el IDE Apache NetBeans 16 (ApacheNetBeans, 2024) y JDK 11. Los experimentos se ejecutaron en un ordenador con un procesador Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz, 20 GB de RAM y sistema operativo de 64 bits, versión Windows 10. En todos los casos se realizan 20 ejecuciones de las heurísticas seleccionadas debido a que no son determinísticas y sus resultados varían, por lo que es necesario realizar pruebas estadísticas.

Se utilizan las nueve heurísticas de construcción implementadas en BHCVRP. Algunas de estas heurísticas requieren de parámetros en su ejecución. A continuación, se presenta la configuración de estos parámetros para las heurísticas que lo requieren:

1. Heurística del Vecino más Cercano con Lista de Candidatos Restringidos (*NN*): propone la construcción de una solución insertando al cliente más cercano dentro de la lista de candidatos restringida al último cliente ya insertado.
 - Tamaño de la lista de candidatos restringidos (*size_RLC*): 3
2. Algoritmo de Ahorros en su versión secuencial (*SaveSeq*): este algoritmo plantea que en una solución dos rutas diferentes pueden ser combinadas formando una nueva ruta. Esta versión del algoritmo propone la construcción de las rutas una a una.
 - λ (*parameter_shape*): 1
3. Algoritmo de Ahorros en su versión paralela (*SaveParall*): esta otra versión del algoritmo tiene la misma filosofía, pero trabaja sobre todas las rutas de forma simultánea.
 - λ (*parameter_shape*): 1
4. Algoritmo de Ahorros basado en *Matching* (*SaveMatch*): esta heurística constituye otra alternativa del algoritmo, pero resuelve un problema de *matching* de peso máximo teniendo en cuenta los pesos de las aristas de cada nodo perteneciente a la lista de clientes en las rutas. Por último, verifica si la unión de dichas rutas es factible, teniendo en cuenta la posible afectación en el futuro.
 - λ (*parameter_shape*): 1
5. Heurística de Inserción Secuencial de Mole & Jameson (*MJ*): esta heurística de inserción utiliza dos medidas para decidir el próximo cliente a insertar en la solución.
 - λ (*c1*): 1, μ (*c2*): 1, y forma de seleccionar el primer cliente a insertar en la ruta (*select_customer_MJ_type*): 1 (el más cercano al depósito).
6. Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (*CMT*): esta heurística de inserción opera en dos fases. En la primera fase se determina la cantidad de rutas a utilizar, junto con un cliente para inicializar cada ruta. En la

segunda fase se crean dichas rutas y se inserta el resto de los clientes en ellas según el costo de inserción.

- λ (*parameter_L*): 1

En el caso de la variante HFVRP se escoge el método de ordenamiento *Descending* y para la variante MDVRP se tiene en cuenta el método de asignación *BestNearest*. Se experimenta con dichas configuraciones por presentar los mejores resultados en la versión anterior de BHCVRP.

3.2 Análisis de los resultados

En la sección Anexos, se presentan las tablas y gráficas que comprenden los resultados de los experimentos 1 y 2. A continuación, se muestran los resultados concluyentes de ambos experimentos.

En el experimento 1 (ver Tabla 8, Tabla 9, Tabla 10, Tabla 11, Tabla 12, Tabla 13, Tabla 14 y Tabla 15, y Figura 5, Figura 6, Figura 7, Figura 8, Figura 9, Figura 10, Figura 11 y Figura 12), se utiliza la prueba estadística no paramétrica de Friedman (Friedman, 1937) para comprobar si los resultados obtenidos con las heurísticas de construcción presentan diferencias significativas. Se utiliza el costo promedio de los problemas como medida para comparar el rendimiento de los nueve algoritmos heurísticos.

En la Tabla 3, se resalta en color azul que para las variantes CVRP y TTRP la heurística *SaveParall* se posiciona como el mejor algoritmo con un *p-value* que sugiere la existencia de diferencias significativas entre los restantes algoritmos heurísticos considerados. Sin embargo, para HFVRP y MDVRP se destaca la heurística *MJ*.

Tabla 3: Rankings de las heurísticas para las cuatro variantes VRP.

Algoritmos	Rankings			
	CVRP	HFVRP	MDVRP	TTRP
<i>Random</i>	8.375	8.5	7.25	8
<i>NN</i>	7	6.1875	7.375	6.75
<i>SaveSeq</i>	2.875	2.5	4.875	2.5
<i>SaveParall</i>	1.875	2.625	2.625	1.125
<i>SaveMatch</i>	5.5	6.25	6.625	4.875
<i>MJ</i>	2.375	2.25	1	2.375
<i>CMT</i>	2.875	2.625	2.375	4.125
<i>Kilby</i>	5.75	6.25	5.625	6.25
<i>Sweep</i>	8.375	7.8125	7.25	9

Respecto al tiempo, para mayor certeza de las afirmaciones anteriores se aplica igualmente la prueba estadística no paramétrica de Friedman (Friedman, 1937). Se utiliza el tiempo promedio de los problemas como medida para comparar el rendimiento de las ocho heurísticas. Como resultado queda *Random* como la heurística que menos tiempo demora en su ejecución para las cuatro variantes de planificación de rutas de vehículos con diferencias significativas con las restantes heurísticas.

En ambos resultados de Friedman, se obtiene un *p-value* inferior a 0.05, por lo que los algoritmos presentan diferencias significativas. Por tanto, se decide realizar un análisis *post-hoc* con los métodos de Li, Finner y Holm. A partir de los resultados obtenidos, se determina que el algoritmo *SaveParall* presenta diferencias significativas con respecto a todos los algoritmos excepto *MJ*, *CMT* y *SaveSeq*.

En el experimento 2 (ver Tabla 16, Tabla 17, Tabla 18, Tabla 19, Tabla 20, Tabla 21, Tabla 22 y Tabla 23, y Figura 13, Figura 14, Figura 15, Figura 16 y Figura 17), se desea comparar el comportamiento de las heurísticas implementadas en la versión anterior de BHCVRP. Con el objetivo de lograr un mayor grado de certeza en la comparación de los algoritmos ya existentes en BHCVRP, se decide aplicar la prueba estadística no

paramétrica de Wilcoxon (Wilcoxon, 1945). Esta prueba se utiliza para determinar si hay diferencias significativas entre los resultados alcanzados por cada par de algoritmos.

Los resultados obtenidos en la Tabla 24 muestran que las heurísticas *Random*, *Sweep*, *SaveSeq*, *SaveParall* y *MJ* tienen un comportamiento similar en ambas versiones de la biblioteca, ya que no presentan diferencias significativas en sus respectivos *p-values*. Sin embargo, la heurística *NN* obtiene un peor rendimiento en cuanto a costo de las soluciones y la heurística *CMT* obtuvo mejoras relevantes.

Los resultados obtenidos en la Tabla 25 muestran que todas las heurísticas, excepto *Random*, tienen un comportamiento similar en ambas versiones de la biblioteca, ya que no presentan diferencias significativas en sus respectivos *p-values*.

Los resultados obtenidos en la Tabla 26 muestran que la heurística *MJ* es la única que permanece con un comportamiento similar en ambas versiones de la biblioteca, ya que no presenta diferencias significativas en sus respectivos *p-values*. La heurística *CMT* obtiene un mejor rendimiento en cuanto a costo de las soluciones, mientras que el resto poseen un rendimiento desfavorable.

Los resultados obtenidos en la Tabla 27 muestran que las heurísticas *Sweep*, *SaveSeq* y *SaveParall* tienen un comportamiento similar en ambas versiones de la biblioteca, ya que no presentan diferencias significativas en sus respectivos *p-values*. Sin embargo, la heurística *NN* obtiene un peor rendimiento en cuanto a costo de las soluciones y el resto obtuvo mejoras relevantes.

En cuanto al tiempo de ejecución, se encuentran diferencias significativas, donde la versión anterior de BHCVRP obtiene mejores resultados (ver Tabla 28, Tabla 29, Tabla 30 y Tabla 31).

Por último, debido a la relevancia de la herramienta OR-Tools, sobre todo para la comunidad en Python, se decide realizar el experimento 3, para determinar la calidad de las soluciones en cuanto a costos y tiempo de ejecución. En las Tabla 32 y Tabla 33, se resumen los mejores resultados en la versión de BHCVRP actual y OR-Tools, para las variantes CVRP y HFVRP. Se escogen solo estas variantes, porque son las comunes para ambos componentes. En cada problema se marca en negrita color azul la mejor

solución obtenida y el menor tiempo de ejecución. En los valores de costo y tiempo de ejecución se consideran dos cifras significativas.

Con el objetivo de lograr un mayor grado de certeza en la comparación de los resultados obtenidos por BHCVRP y OR-Tools, se decide aplicar la prueba estadística no paramétrica de Wilcoxon (Wilcoxon, 1945) como se aprecia en la Tabla 34. Esta prueba se utiliza para determinar si hay diferencias significativas entre los resultados alcanzados por cada componente de software.

Los resultados obtenidos con la prueba de Wilcoxon muestran diferencias significativas en cuanto al costo en distancia entre ambos componentes de software. Se obtienen notablemente mejores soluciones con BHCVRP.

En cuanto al tiempo de ejecución, en las gráficas de las Figura 18 y Figura 19, se muestra el rendimiento por cada par de soluciones (BHCVRP y OR-Tools). Como se puede apreciar en los dos casos, OR-Tools obtiene mejores tiempos de ejecución.

Por último, se decide aplicar la prueba estadística no paramétrica de Wilcoxon (Wilcoxon, 1945) (ver Tabla 35) para comparar los tiempos promedios de las ocho instancias para CVRP y HFVRP, en ambas herramientas.

A partir de los resultados alcanzados se puede arribar a las siguientes conclusiones: existen diferencias significativas en cuanto a tiempo de ejecución para ambas variantes. La herramienta OR-Tools obtiene los resultados en considerablemente menor tiempo que BHCVRP.

Conclusiones

Después de dar cumplimiento a los objetivos trazados en este trabajo y a partir de los resultados alcanzados se puede concluir que los Problemas de Planificación de Rutas de Vehículos son problemas de optimización combinatoria y una buena alternativa para solucionarlos son los algoritmos heurísticos y metaheurísticos. Las heurísticas de construcción presentan buenos resultados para estos problemas y son aplicables como punto de partida en el contexto de las metaheurísticas. Actualmente, existen escasos componentes de software que implementen métodos heurísticos para resolver variantes de VRP. De los componentes de software estudiados solo BICIAM, VRPH, BHCVRP, JSprit, VROOM y OR-Tools resuelven problemas de planificación de rutas de vehículos. Sin embargo, solo Hipster y BHCVRP implementan heurísticas de construcción exclusivamente. Es importante destacar que este último resuelve solamente las variantes CVRP, MDVRP, HFVRP y TTRP.

La nueva versión de BHCVRP resuelve algunas de las deficiencias encontradas en la versión anterior. Se incorporan dos nuevas heurísticas de construcción: el Algoritmo de Ahorro basado en *Matching* y la Heurística de Inserción de Kilby. Además, se plasman un conjunto de mejoras en la arquitectura y funcionamiento de la biblioteca como son: la creación de la primera versión que cubre los lenguajes de programación Python y Java, el tratamiento de excepciones propias relacionadas con la distancia, capacidad de los vehículos, velocidad y tiempo de ejecución, así como la aplicación del patrón de diseño *Template*.

Los resultados alcanzados en los experimentos haciendo uso de la prueba estadística de Friedman permite afirmar que en las variantes CVRP y TTRP, la heurística *SaveParallel* es la de mejor rendimiento. Por otra parte, para las variantes HFVRP y MDVRP, la heurística más apropiada es *MJ*. En cuanto al tiempo de ejecución, se evidencia que la heurística *MJ* es la que emplea mayor cantidad, pero alcanza mejores resultados. Sin embargo, el Método Aleatorio obtiene un tiempo de ejecución pequeño con los peores resultados para todos los casos. A partir de la comparación de las dos versiones de BHCVRP se aprecia que la mayoría de las heurísticas mantienen un comportamiento similar en las cuatro variantes VRP. Aunque, es importante destacar que la heurística

NN de manera general fue desfavorable y *CMT* mejoró su rendimiento. Por último, en el experimento comparativo realizado con la herramienta OR-Tools, BHCVRP obtiene mejores resultados en cuanto a costo de las soluciones a pesar de tomar mayor cantidad de tiempo.

Recomendaciones

A partir del desarrollo de este trabajo se identifican un conjunto de aspectos a ser considerados en futuras iteraciones de este proceso de investigación. A continuación, se presentan las principales recomendaciones que se derivan de este trabajo:

- Añadir las modificaciones realizadas en cuanto a arquitectura y código fuente, en la versión de BHCVRP en Python, a la versión existente en Java.
- Incorporar otras variantes de VRP que reflejen nuevas características de la vida real. Por ejemplo, SBRP, OVRP, VRPTW, entre otras.
- Extender la biblioteca a partir de la incorporación de nuevas heurísticas de construcción, como puede ser la Heurística de Asignación Generalizada de Fisher y Jaikumar, el Algoritmo de Pétalos, entre otras.
- Con el objetivo de proporcionar mayor flexibilidad a la biblioteca permitir al usuario el uso de pasos de post-optimización a su conveniencia.
- Incorporar nuevas excepciones propias, por ejemplo, para las heurísticas que poseen parámetros como las versiones del Algoritmo de Ahorros, las heurísticas de inserción *Mole & Jameson*, *CMT* y *Kilby*.
- Respecto a la variante MDVRP sería conveniente incorporar nuevos métodos de asignación y admitir por parte del usuario una asignación predeterminada.
- Mejorar el funcionamiento de heurísticas como *NN*, *Kilby* y *SaveMatch*, así como los resultados para la variante MDVRP. Además, incorporar experimentos con el resto de las configuraciones que brindan las variante HFVRP (orden *Ascending* y *Random*) y MDVRP (todas las asignaciones que brinda BHAVRP, excepto *BestNearest*).
- Realizar nuevos experimentos y compararlos con los resultados publicados en la literatura.

Referencias bibliográficas

- Accorsi, L., & Vigo, D. (2020). A hybrid metaheuristic for single truck and trailer routing problems. *Transportation Science*, 54(5), 1351-1371.
- Alba, E. (2014). Networking and emerging optimization. Retrieved from <http://neo.lcc.uma.es/vrp/vrp-instances/>
- Alba, E., Almeida, F., Blesa, M., Cabeza, J., Cotta, C., Díaz, M., . . . Luna, J. (2002). *MALLBA: A library of skeletons for combinatorial optimisation*. Paper presented at the European Conference on Parallel Processing.
- Algarra, L., & Pelta, D. A. (2012). DVRP-OCR: Un Método Cooperativo Basado en Heurísticas Simples para el DVRP VII Congreso Español sobre Metaheurísticas, Algoritmos Evolutivos y Bioinspirados. Albacete, Spain.
- ApacheNetBeans. (2024). Welcome to Apache NetBeans. Retrieved from <https://netbeans.apache.org/front/main/index.html>
- Applegate, D. L. (2006). *The traveling salesman problem: a computational study* (Vol. 17): Princeton university press.
- Arifta, E., & Rakhmawati, F. (2023). Analysis of Book Distribution Routes Using the Capacity Vehicle Routing Problem (CVRP) Method Using the Sweep Algorithm. *Sinkron: jurnal dan penelitian teknik informatika*, 8(1), 360-367.
- Baldacci, R., Toth, P., & Vigo, D. (2010). Exact algorithms for routing problems under vehicle capacity constraints. *Annals of Operations Research*, 175, 213-245.
- Calderón Calderón, O. E. (2020). *Diseño e implementación de modelo matemático para la optimización del recorrido de camiones recolectores de residuos sólidos en el municipio de León*.
- Capilla Sevilla, R. (2022). Temas Diseño y Arquitectura del Software.
- Chao, I. M. (2002). A tabu search method for the truck and trailer routing problem. *Computers & Operations Research*, 29, 33-51.
- Clancey, W. J. (1985). Heuristic classification. *Artificial intelligence*, 27(3), 289-350.

Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations research*, 12(4), 568-581.

Cordeau, J.-F., & décisions, G. d. é. e. d. r. e. a. d. (2000). *The VRP with time windows*: Citeseer.

Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. *Management science*, 6(1), 80-91.

Daza, J. M., Montoya, J. R., & Narducci, F. (2009). Resolución del problema de enrutamiento de vehículos con limitaciones de capacidad utilizando un procedimiento metaheurístico de dos fases. *Revista EIA*(12), 23-38.

Díaz-Subi, L. (2016). *Nueva versión de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos*. (Tesis de diploma), Instituto Superior Politécnico José Antonio Echeverría (CUJAE).

Didier, F., Perron, L., Mohajeri, S., Gay, S. A., Cuvelier, T., & Furnon, V. (2023). OR-Tools' vehicle routing solver: a generic constraint-programming solver with heuristic search for routing problems.

Eckel, B. (2003). *Thinking in Patterns with Java*: President, MindView, Inc.

Eksioglu, B., Vural, A. V., & Reisman, A. (2009). The vehicle routing problem: A taxonomic review. *Computers & industrial engineering*, 57(4), 1472-1483.

Fisher, M. L., & Jaikumar, R. (1981). A generalized assignment heuristic for vehicle routing. *Networks*, 11(2), 109-124.

Florios, K., & Mavrotas, G. (2014). Generation of the exact pareto set in multi-objective traveling salesman and set covering problems. *Applied Mathematics and Computation*, 237, 1-19.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200), 674-701.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: elements of reusable object-oriented software*: Pearson Deutschland GmbH.

Gillett, B. E., & Miller, L. R. (1974). A heuristic algorithm for the vehicle-dispatch problem. *Operations research*, 22(2), 340-349.

Golden, B., Wang, X., & Wasil, E. (2023). The evolution of the Vehicle Routing Problem—A survey of VRP research and practice from 2005 to 2022 *The Evolution of the Vehicle Routing Problem: A Survey of VRP Research and Practice from 2005 to 2022* (pp. 1-64): Springer.

González Restrepo, D. A., & Gómez Veloza, D. Y. (2019). Solución al problema de ruteo de vehículos con entregas y recogidas aplicando el algoritmo de pétalos y la heurística del vecino más cercano.

Groër, C., Golden, B., & Wasil, E. (2010). A library of local search heuristics for the vehicle routing problem. *Mathematical Programming Computation*, 2, 79-101.

Hochba, D. S. (1997). Approximation algorithms for NP-hard problems. *ACM Sigact News*, 28(2), 40-52.

Karkula, M., Duda, J., & Skalna, I. (2019). Comparison of capabilities of recent open-source tools for solving Capacitated Vehicle Routing Problems with Time Windows. *Carpathian Logistics Congress*. Retrieved from

Konstantakopoulos, G. D., Gayialis, S. P., & Kechagias, E. P. (2022). Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. *Operational research*, 22(3), 2033-2062.

Kumar, S. N., & Panneerselvam, R. (2012). A survey on the vehicle routing problem and its variants. *Intelligent Information Management*, 04(03). doi:10.4236/iim.2012.43010

Laporte, G. (1992). The vehicle routing problem: An overview of exact and approximate algorithms. *European journal of operational research*, 59(3), 345-358.

Lima, E. (2017). *CommuteVRP: otimização de serviços privados de transporte contínuo*. Paper presented at the Anais do XIII Simpósio Brasileiro de Sistemas de Informação.

Machado, M. (2019). *Biblioteca de Heurísticas de Asignación para el Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos*. (Trabajo de diploma para

optar por el título de Ingeniería en Informática), Universidad Tecnológica de La Habana "José Antonio Echeverría" (CUJAE), La Habana.

Mahmoud, A., Chouaki, T., Hörl, S., & Puchinger, J. (2022). Extending JSprit to solve electric vehicle routing problems with recharging. *Procedia Computer Science*, 201, 289-295.

Maischberger, M. (2011). COIN-OR METSlib a Metaheuristics Framework in Modern C++. *Firenze University*. Recuperado de <https://projects.coin-or.org/metslib>.

Martí, R. (2003). Procedimientos metaheurísticos en optimización combinatoria. *Matemàtiques, Universitat de València*, 1(1), 3-62.

Máximo, V. R., Cordeau, J.-F., & Nascimento, M. C. (2022). An adaptive iterated local search heuristic for the Heterogeneous Fleet Vehicle Routing Problem. *Computers & Operations Research*, 148, 105954.

Medina, L. B. R., La Rota, E. C. G., & Castro, J. A. O. (2011). Una revisión al estado del arte del problema de ruteo de vehículos: Evolución histórica y métodos de solución. *Ingeniería*, 16(2), 35-55.

Mole, R., & Jameson, S. (1976). A sequential route-building algorithm employing a generalised savings criterion. *Journal of the Operational Research Society*, 27(2), 503-511.

Morsidi, F., & Panessai, I. Y. (2023). Overview of the Integral Impact of MDVRP Routing Variables on Routing Heuristics. *Applied Information Technology And Computer Science*, 4(1), 1723-1738.

Nerey, R. (2017). *Herramienta para la generación de rutas en el problema del transporte obrero: Trans-o*. (Tesis de Diploma), Universidad Tecnológica de La Habana "José Antonio Echeverría" (CUJAE), La Habana.

Nurcahyo, G. W., Alias, R. A., Shamsuddin, S. M., & Sap, M. N. M. (2002). Sweep algorithm in vehicle routing problem for public transport. *Jurnal Antarabangsa Teknologi Maklumat*, 2, 51-64.

Offermann, P., Blom, S., Schönherr, M., & Bub, U. (2010). *Artifact types in information systems design science—a literature review*. Paper presented at the Global Perspectives on Design Science Research: 5th International Conference, DESRIST 2010, St. Gallen, Switzerland, June 4-5, 2010. Proceedings. 5.

Olivera, A. (2004). Heurísticas para problemas de ruteo de vehículos. *Reportes Técnicos 04-08*.

Papadimitriou, C. H., & Steiglitz, K. (1998). *Combinatorial optimization: algorithms and complexity*: Courier Corporation.

PAREJO, M., DÍAZ, L., PARRA, L., VILLARREAL, H., FABREGAS, J., PALENCIA, A., & VELILLA, W. (2020). Desarrollo de un algoritmo para seleccionar la ubicación de puntos de abastecimiento minimizando los tiempos y costos de transporte. *Revista Espacios*, 41(17), 1-15.

Pérez-Pérez, A. C., Sánchez-Ansola, E., & Rosete-Suárez, A. (2021). A metaheuristic solution for the school bus routing problem with homogeneous fleet and bus stop selection. *Ingeniería*, 26(2), 233-253.

Pérez, N. (2022). *Incorporación de algoritmos de agrupamiento en la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos*. (Trabajo de diploma para optar por el título de Ingeniería en Informática), Universidad Tecnológica de La Habana "José Antonio Echeverría" (CUJAE), La Habana.

Piqueras, V. Y. (2002). *Optimización heurística económica aplicada a las redes de transporte del tipo VRPTW*. (Phd), Universidad Politécnica de Valencia, Valencia, España.

PyCharm. (2024, 22 de agosto de 2024). Install PyCharm | PyCharm Documentation. Retrieved from <https://www.jetbrains.com/help/pycharm/installation-guide.html>

Reynoso, C. B. (2004). Introducción a la Arquitectura de Software. *Universidad de Buenos Aires*, 33.

- Rita M. Newton, & Thomas, W. (1969). Design of School Bus Routes by Computer. *Socio-Economic Planning Sciences*, 3, 75-85. doi:10.1016/0038-0121(69)90051-2
- Romanycia, M. H., & Pelletier, F. J. (1985). What is a heuristic? *Computational intelligence*, 1(1), 47-58.
- Sommerville, I., & Wesley, A. (2011). Introducción a la Ingeniería de Software. *I. Sommerville, Ingeniería de Software. Pearson Educación.*
- Souza, V. R. d. (2020). Uma análise do framework OptaPlanner aplicado ao problema de empacotamento unidimensional. *Research Gate*. Retrieved from Research Gate website:
- Talbi, E.-G. (2009). *Metaheuristics: from design to implementation* (Vol. 74). Hoboken, New Jersey: John Wiley & Sons, Inc.
- Tangarife Álvarez, A. F. (2017). Revisión del estado del arte del problema de ruteo abierto (OVRP).
- Toth, P., & Vigo, D. (2002). *The vehicle routing problem*: SIAM.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80-83.
- Zirour, M. (2008). Vehicle routing problem: models and solutions. *Journal of Quality Measurement and Analysis JQMA*, 4(1), 205-218.

Anexos

Tabla 4: Descripción de las instancias CVRP.

Problema	Total de clientes	Total de vehículos	Capacidad de los vehículos
1	50	5	200
2	75	9	250
3	75	3	480
4	100	5	400
5	360	5	500
6	200	50	200
7	400	100	200
8	600	150	200

Tabla 5: Descripción de las instancias MDVRP.

Problema	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
1	50	4	4	80
2	100	2	5	200
3	160	4	5	60
4	360	9	5	60
5	288	4	6	175
6	249	4	8	500
7	50	4	2	160
8	144	4	3	190

Tabla 6: Descripción de las instancias HFVRP.

Datos originales			Flota de Vehículos	
Problema	Total de clientes	Total de vehículos	Total	Capacidad
1	50	5	1	200
			1	400
			1	100
			1	250
			1	50
2	75	9	2	250
			2	100
			2	455
			1	90
			1	50
			1	500
3	75	3	2	470
			1	500
4	100	5	1	400
			1	100
			1	200
			2	650
5	360	5	3	500
			1	400
			1	600
6	200	50	25	240
			25	160
7	400	100	50	320
			50	80
8	600	150	75	220
			75	180

Tabla 7: Descripción de las instancias TTRP.

Problema	Clientes			Camiones		Remolques	
	Total	VC	TC	Total	Capacidad	Total	Capacidad
1	50	38	12	5	100	3	100
2	75	38	37	9	100	5	100
3	100	50	50	8	150	4	100
4	120	60	60	7	150	4	100
5	100	50	50	10	150	5	100
6	199	100	99	17	150	9	100
7	50	13	37	5	100	3	100
8	150	38	112	12	150	6	100

Tabla 8: Resumen de los resultados en las ocho instancias CVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1470.02 ₍₄₎	1537.38	0.0019	1326.55 ₍₄₎	1474.08	0.1186	558.89 ₍₄₎	576.60	0.5448	561.46 ₍₄₎	568.86	1.2490	953.83 ₍₅₎	1005.35	0.6707
2	2190.94 ₍₆₎	2392.85	0.0535	2195.57 ₍₆₎	2249.47	0.3844	787.61 ₍₆₎	815.77	1.2557	733.2 ₍₆₎	750.04	3.1785	1168.18 ₍₆₎	1260.79	2.1261
3	2199.51 ₍₃₎	2344.06	0.0456	2081.68 ₍₃₎	2214.83	0.4135	674.88 ₍₃₎	702.43	1.2211	669.99 ₍₃₎	673.47	2.9397	1133.91 ₍₃₎	1203.82	2.2798
4	3003.18 ₍₄₎	3051.01	0.0584	2577.62 ₍₄₎	2729.04	0.8748	855.49 ₍₄₎	878.09	2.7445	785.84 ₍₄₎	786.31	6.4984	1679.47 ₍₄₎	1748.28	4.2555
5	42857.41 ₍₃₎	45456.2	0.1443	20577.08 ₍₃₎	22842.9	16.024	6134.72 ₍₄₎	6215.2	94.805	5386.65 ₍₄₎	5386.7	309.6680	33783.06 ₍₅₎	35587.11	180.0369
6	14447.08 ₍₁₈₎	14776.5	0.1187	13590.27 ₍₁₈₎	13947.04	5.0068	2885.78 ₍₁₈₎	2981.08	16.2967	2689.54 ₍₁₉₎	2689.54	47.8148	6360.92 ₍₁₈₎	6442.67	33.7310
7	42733.71 ₍₃₇₎	43345.77	0.0544	40768.37 ₍₃₇₎	41755.41	3.4151	8129.33 ₍₃₆₎	8426.49	12.167	7261.48 ₍₃₈₎	7261.48	496.4777	14464.05 ₍₃₆₎	14597.07	236.7240
8	89891.21 ₍₅₆₎	89997.35	0.188	85460.28 ₍₅₆₎	86530.78	6.434	16998.52 ₍₅₆₎	17402.21	451.87	13678.65 ₍₅₆₎	13678.65	951.639	29423.53 ₍₅₆₎	29516.93	889.3156

Tabla 9: Resumen de los resultados en las ocho instancias CVRP (II).

Problema	MJ			CMT			Kilby			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	597.38 ₍₄₎	612.15	2.4681	598.96 ₍₄₎	640.49	0.5086	1044.96 ₍₅₎	1060.82	1.1911	1435.45 ₍₄₎	1514.74	0.2259
2	801.6 ₍₆₎	816.92	8.7697	806.83 ₍₆₎	854.40	1.3093	1199.09 ₍₉₎	1255.48	5.9780	2293.26 ₍₆₎	2386.57	0.7546
3	666.86 ₍₃₎	676.99	15.7553	800.91 ₍₃₎	881.26	1.7931	1824.51 ₍₃₎	1856.99	2.6769	2247.21 ₍₃₎	2354.26	0.8910
4	814.49 ₍₄₎	835.15	36.1031	1005.45 ₍₄₎	1100.78	3.3230	1532.82 ₍₅₎	1570.03	7.3232	3043.83 ₍₄₎	3252.77	1.9394
5	6499.42 ₍₄₎	6534.7	337.7738	16421.59 ₍₄₎	17693.7	31.3677	31942.41 ₍₅₎	32017.5	311.9699	40998.82 ₍₄₎	44109.3	87.2194
6	2689.13 ₍₁₈₎	2875.5	125.1227	2391.66 ₍₁₈₎	2454.23	9.7152	6231.99 ₍₅₀₎	6316.00	445.3573	14552.85 ₍₁₈₎	14925.9	13.2280
7	7256.86 ₍₃₆₎	7495.7	63.1086	6239.59 ₍₃₆₎	6467.23	136.5664	18854.77 ₍₃₈₎	18904.3	304.4157	42050.21 ₍₃₇₎	43031.5	10.8614
8	15288.58 ₍₅₆₎	15604.8	214.7608	13386.91 ₍₅₆₎	13421.6	62.2868	40000.05 ₍₅₇₎	42390.56	500.9123	84491.08 ₍₅₆₎	85832.8	449.4740

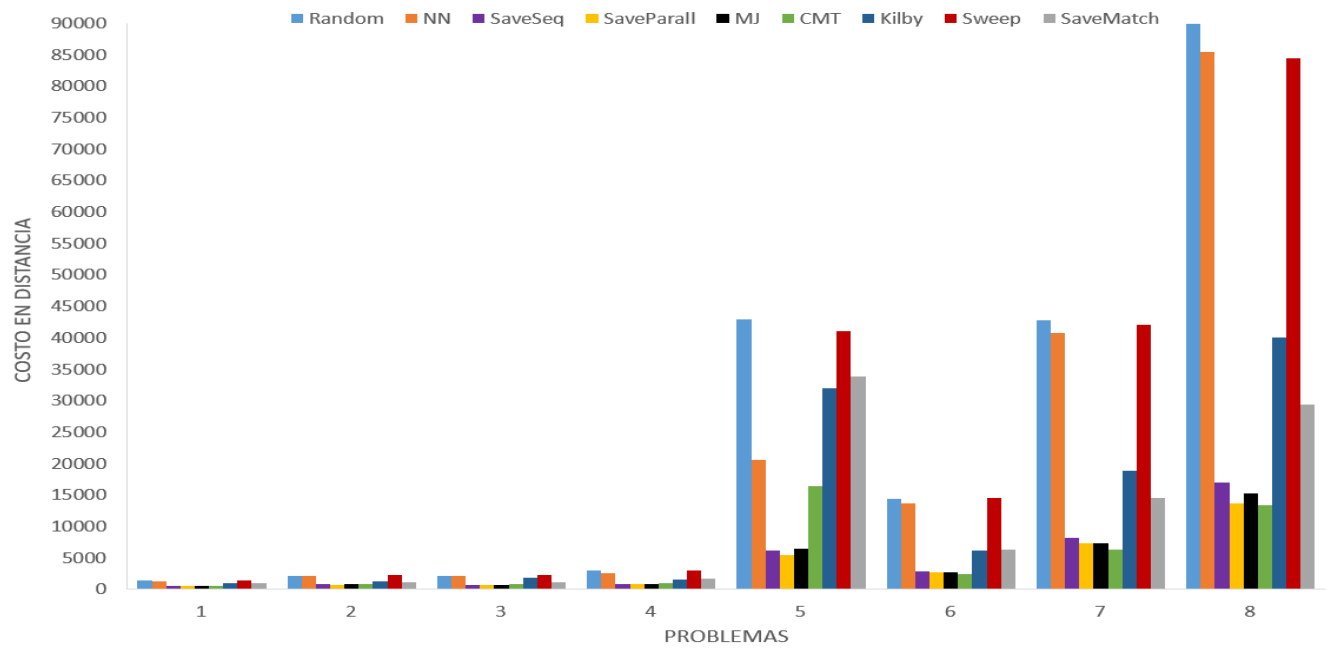


Figura 5: Gráfica de barras para el costo de las heurísticas en la variante CVRP.

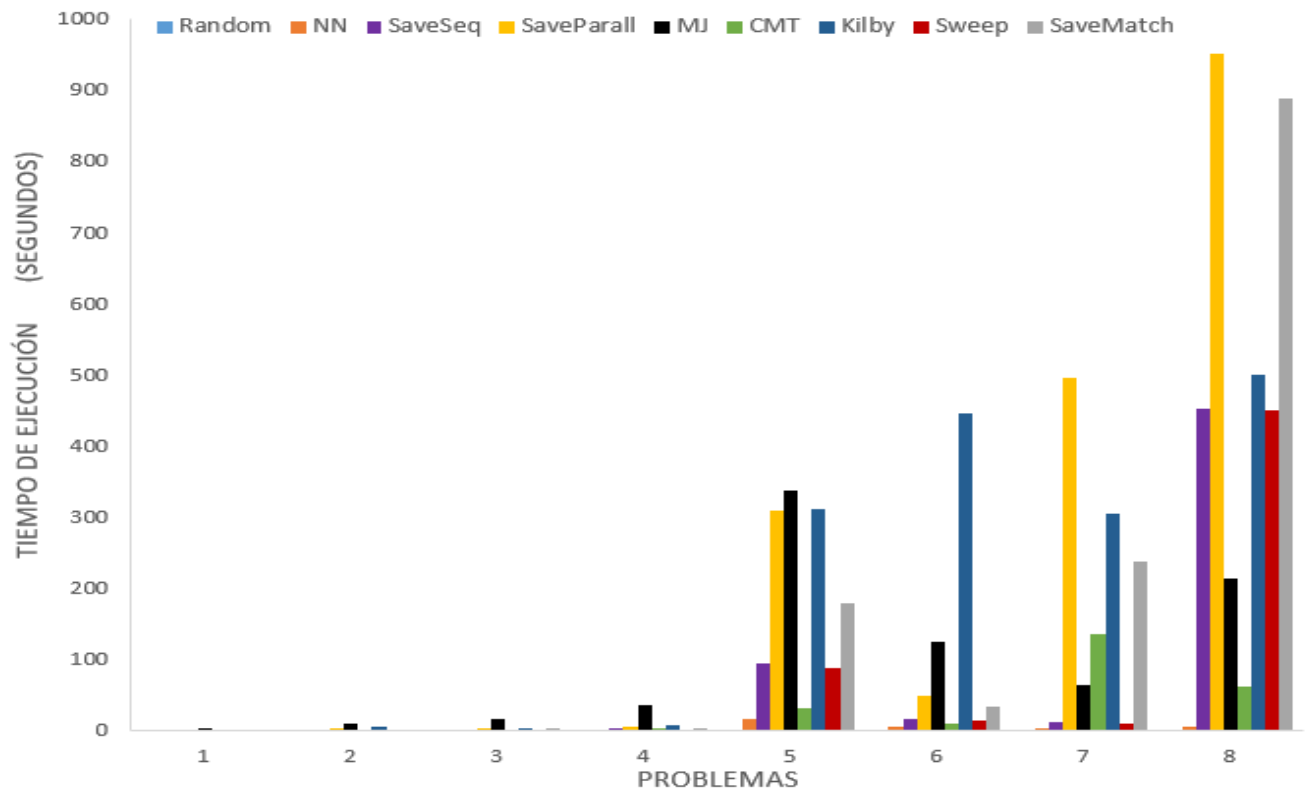


Figura 6: Gráfica de barras para el tiempo de ejecución de las heurísticas en la variante CVRP.

Tabla 10: Resumen de los resultados en las ocho instancias HFVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1058.99 ₍₃₎	1152.25	0.0351	806.62 ₍₃₎	917.40	0.1540	550.34 ₍₄₎	566.53	0.4820	474.33 ₍₂₎	497.99	1.0935	839.36 ₍₃₎	922.51	1.7534
2	1914.54 ₍₅₎	2061.52	0.0471	1604.56 ₍₅₎	1696.16	0.4395	802.7 ₍₆₎	835.85	1.4416	851.96 ₍₇₎	851.96	1.181	1133.9 ₍₃₎	1185.49	7.1024
3	1444.78 ₍₄₎	1594.74	0.0456	1130.49 ₍₄₎	1171.20	0.4109	666.31 ₍₃₎	698.18	1.3554	653.83 ₍₃₎	658.68	4.1937	1170.71 ₍₃₎	1213.99	3.7284
4	2983.77 ₍₃₎	3123.60	0.0721	1899.58 ₍₃₎	1960.17	0.8415	768.28 ₍₃₎	774.38	2.6611	982.69 ₍₄₎	990.28	13.602	1613.45 ₍₃₎	1527.92	9.8802
5	43889.95 ₍₃₎	44459.46	0.2249	21942.66 ₍₃₎	22355.98	33.670	6182.04 ₍₄₎	6255.36	24.611	7245.90 ₍₄₎	7277.43	41.33	35223.4 ₍₄₎	35622.27	425.6863
6	13510.74 ₍₁₄₎	13923.44	0.2639	12729.4 ₍₁₄₎	13096.29	6.1536	2597.15 ₍₁₅₎	2680.11	20.9691	2525.17 ₍₁₅₎	2620.46	102.1874	6320.81 ₍₁₅₎	6320.81	572.2572
7	39048.76 ₍₂₂₎	39743.58	0.2715	36842.51 ₍₂₂₎	38047.66	35.7714	5787.86 ₍₂₃₎	5934.57	126.7698	5955.38 ₍₅₎	5970.02	4.4937	13838.25 ₍₂₃₎	13838.25	8460.7132
8	85067.42 ₍₅₁₎	86007.76	0.5714	81370.56 ₍₅₁₎	83576.92	119.4991	15294.68 ₍₅₁₎	15882.32	220.9266	16967.84 ₍₈₎	18945.3	7.9899	31177.03 ₍₅₁₎	31177.03	40309.355

Tabla 11: Resumen de los resultados en las ocho instancias HFVRP (II).

Problema	MJ			CMT			Kilby			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	566.31 ₍₄₎	582.13	3.9872	587.91 ₍₄₎	625.13	0.6129	1043.87 ₍₄₎	1065.77	1.3930	1029.56 ₍₃₎	1121.32	0.2746
2	799.87 ₍₆₎	816.30	9.3635	802.23 ₍₆₎	851.58	1.3590	1211.89 ₍₉₎	1255.69	5.7586	1883.38 ₍₅₎	2067.59	0.8829
3	670.57 ₍₃₎	698.27	16.6713	848.13 ₍₃₎	913.49	1.7733	1826.66 ₍₃₎	1896.52	3.0008	1343.62 ₍₂₎	1578.88	0.8762
4	782.9 ₍₃₎	784.41	32.1689	1180.74 ₍₃₎	1242.09	3.4130	1533.05 ₍₅₎	1556.23	7.3406	2121.06 ₍₃₎	2123.70	1.8406
5	6266.98 ₍₄₎	6395.63	6855.8267	16782.7 ₍₄₎	18917.08	112.6921	31860.28 ₍₅₎	32125.54	85.1255	43438.99 ₍₃₎	44260.40	80.9879
6	2340.17 ₍₁₅₎	2408.97	371.9073	2118.76 ₍₁₅₎	2163.73	19.9983	6148.38 ₍₅₀₎	6231.82	490.645	12729.4 ₍₁₄₎	13096.29	6.1536
7	5442.21 ₍₂₃₎	5503.95	217.3151	4705.29 ₍₂₃₎	4905.91	121.9479	18757.2 ₍₁₀₀₎	18890.44	6833.156	39203.75 ₍₂₂₎	40799.19	213.4071
8	13769.09 ₍₅₁₎	13940.0	183.9995	11564.1 ₍₅₁₎	12345.32	1003.677	36976.48 ₍₁₅₀₎	37395.33	1392.285	85733.38 ₍₅₁₎	86365.48	336.3865

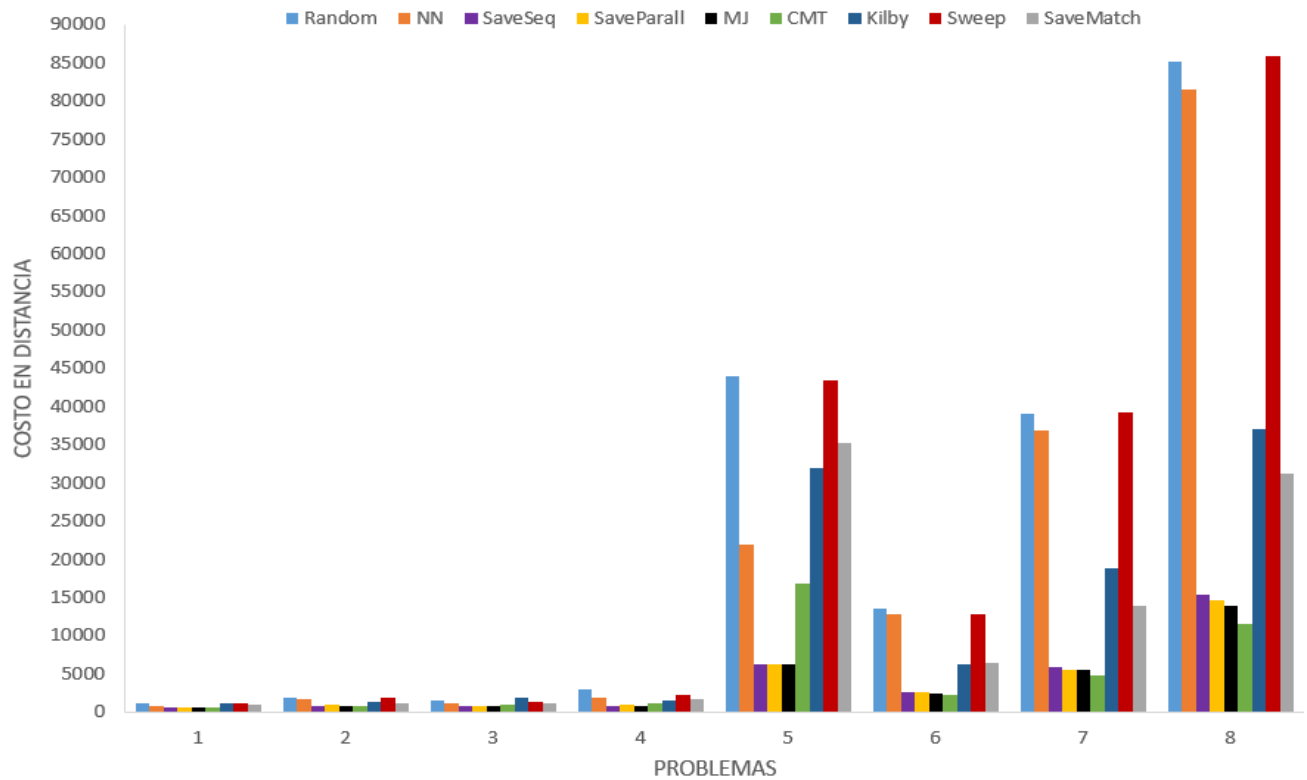


Figura 7: Gráfica de barras para el costo de las heurísticas en la variante HFVRP.

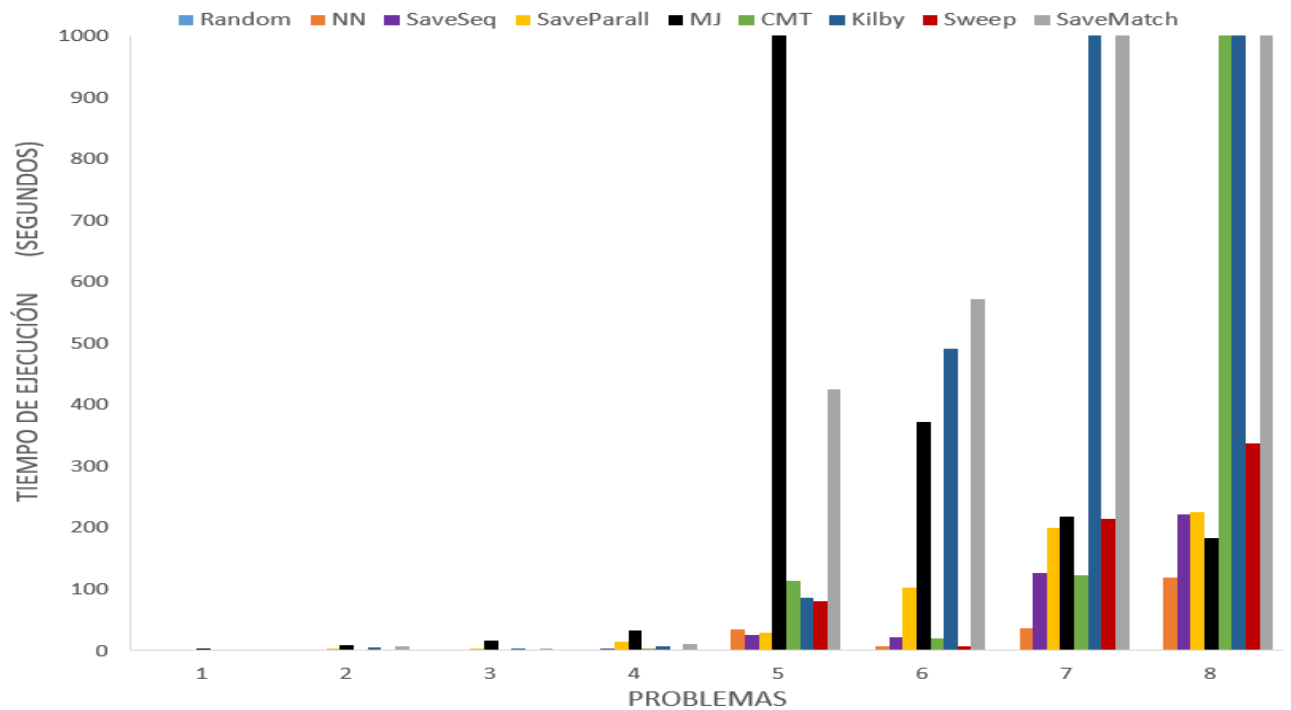


Figura 8: Gráfica de barras para el tiempo de ejecución de las heurísticas en la variante HFVRP.

Tabla 12: Resumen de los resultados en las ocho instancias MDVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	954.14 ₍₁₂₎	1094.55	0.0900	963.18 ₍₁₂₎	1014.82	0.0828	1022.54 ₍₁₂₎	1062.98	0.2845	786.07 ₍₁₃₎	786.07	0.4551	1418.77 ₍₁₂₎	1418.77	0.2841
2	2407.18 ₍₉₎	2552.84	0.0788	2269.18 ₍₉₎	2359.61	0.5166	1099.85 ₍₉₎	1144.25	1.4595	974.44 ₍₉₎	974.44	3.3595	1293.48 ₍₁₀₎	1406.76	2.2098
3	8031.91 ₍₁₆₎	8215.06	0.1177	8200.01 ₍₁₆₎	8386.06	0.8904	5562.36 ₍₁₆₎	5698.81	2.7322	3318.6 ₍₁₆₎	3318.6	4.7684	8403.63 ₍₁₆₎	8403.63	4.2784
4	18571.65 ₍₃₇₎	18917.24	0.3887	18718.21 ₍₃₆₎	19071.19	3.3262	18709.53 ₍₃₆₎	18886.34	12.5386	7466.85 ₍₃₆₎	7466.85	16.923	20070.49 ₍₃₆₎	20070.49	20.0164
5	10754.42 ₍₂₃₎	11242.51	0.2773	8704.52 ₍₂₃₎	8965.72	3.4387	4930.64 ₍₂₃₎	5039.68	13.976	4024.24 ₍₂₃₎	4024.24	23.3302	6286.74 ₍₂₂₎	6659.06	16.0464
6	15116.31 ₍₂₇₎	15516.12	0.2783	13267.24 ₍₂₈₎	13761.88	2.5329	8102.38 ₍₂₇₎	8420.09	9.2313	6021.26 ₍₂₇₎	6021.26	15.7345	10028.45 ₍₂₆₎	10448.35	8.4764
7	853.62 ₍₇₎	936.43	0.0925	877.89 ₍₇₎	925.82	0.1336	735.19 ₍₇₎	755.19	0.2717	602.23 ₍₇₎	602.23	0.3698	792.08 ₍₇₎	872.21	2.1052
8	5677.25 ₍₁₂₎	6009.17	0.1951	4798.05 ₍₁₁₎	4966.42	0.6425	2686.37 ₍₁₁₎	2761.28	2.3026	2639.18 ₍₁₂₎	2639.18	3.6211	3726.6 ₍₁₂₎	3911.75	4.2291

Tabla 13: Resumen de los resultados en las ocho instancias MDVRP (II).

Problema	MJ			CMT			Kilby			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	645.99 ₍₈₎	645.99	0.3784	712.66 ₍₁₂₎	748.74	0.2281	1157.68 ₍₁₆₎	1157.68	0.2965	920.69 ₍₁₂₎	1027.11	0.1339
2	790.09 ₍₆₎	803.09	9.9856	981.77 ₍₉₎	1030.56	1.8661	1406.47 ₍₁₀₎	1434.98	4.9874	2474.59 ₍₉₎	2572.76	0.5599
3	2690.26 ₍₁₂₎	2790.24	18.0643	3603.04 ₍₁₆₎	3752.61	3.4766	4716.17 ₍₂₀₎	4756.19	8.8169	7963.48 ₍₁₆₎	8219.78	0.6854
4	6339.63 ₍₂₇₎	6392.52	82.2219	8419.52 ₍₃₆₎	8677.32	16.2640	10813.19 ₍₄₅₎	10937.99	43.0887	18357.21 ₍₃₆₎	18875.29	1.5296
5	3664.36 ₍₁₉₎	3736.03	107.2054	3936.36 ₍₂₃₎	4139.97	15.0633	6876.3 ₍₂₄₎	7028.82	65.3264	10795.79 ₍₂₃₎	11196.25	2.6812
6	4952.36 ₍₂₂₎	5133.44	48.9223	5308.3 ₍₂₇₎	5427.71	9.4167	8611.26 ₍₃₂₎	8769.58	54.9366	14979.95 ₍₂₇₎	15591.27	2.0606
7	351.46 ₍₃₎	367.82	0.7900	586.94 ₍₇₎	635.25	0.3275	839.98 ₍₈₎	862.37	0.4989	873.73 ₍₇₎	947.57	0.0618
8	1834.98 ₍₇₎	1894.69	17.2066	2586.14 ₍₁₁₎	2761.5	3.4065	3784.55 ₍₁₂₎	3890.21	5.7184	5763.37 ₍₁₁₎	5966.67	0.4829

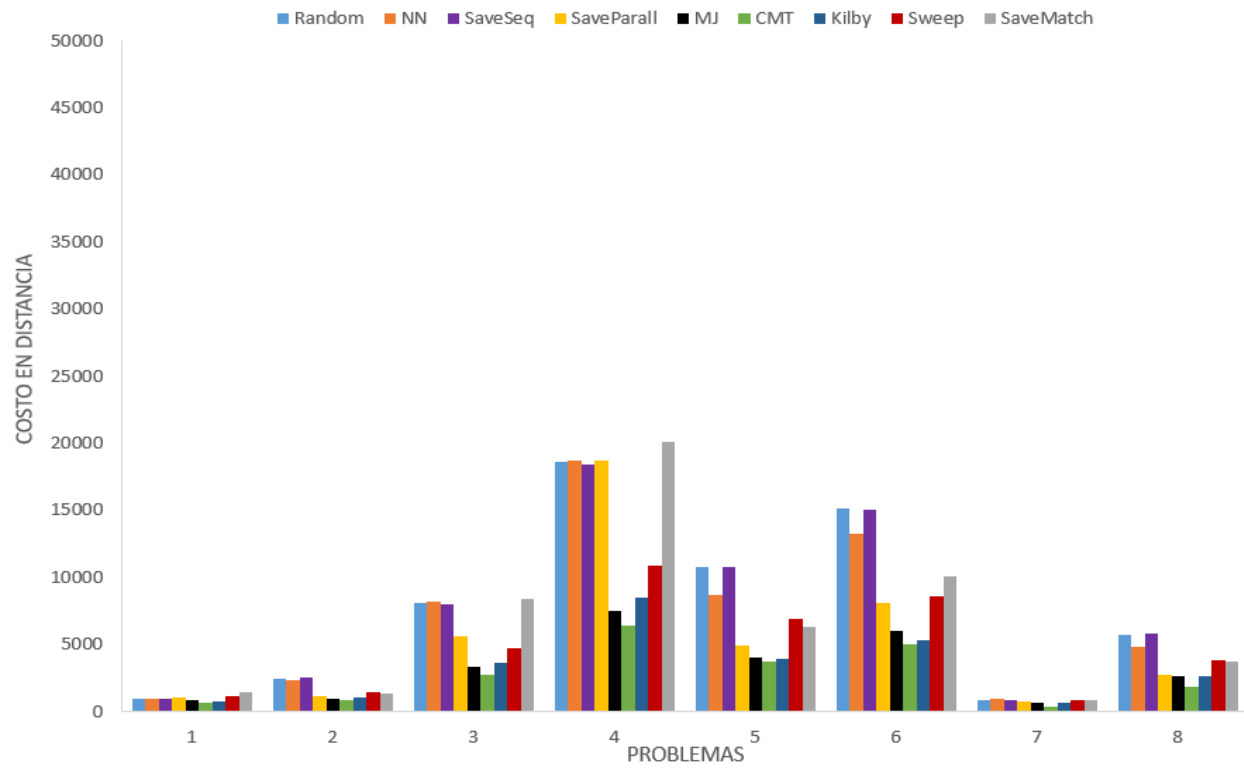


Figura 9: Gráfica de barras para el costo de las heurísticas en la variante MDVRP.

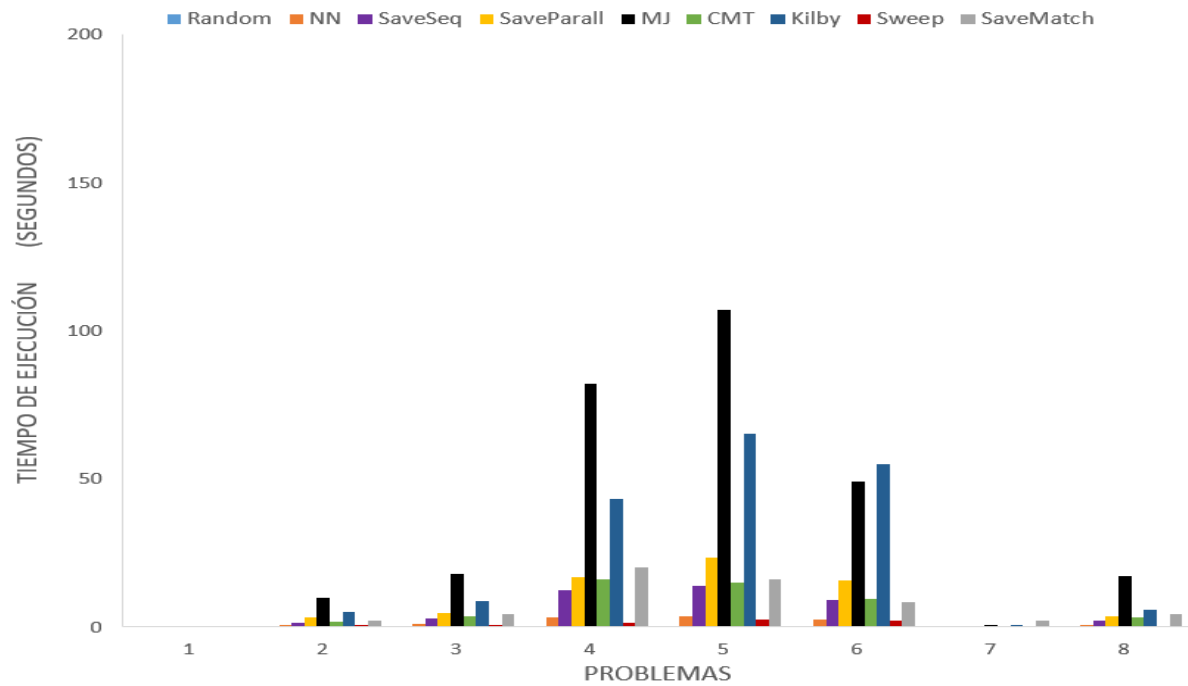


Figura 10: Gráfica de barras para el tiempo de ejecución de las heurísticas en la variante MDVRP.

Tabla 14: Resumen de los resultados en las ocho instancias TTRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1449.5 ₍₆₎	1561.81	0.045	1393.7 ₍₆₎	1524.49	0.1817	605.8 ₍₁₎	610.36	0.417	570.14 ₍₄₎	580.79	0.5273	935.6 ₍₄₎	956.92	0.4926
2	2402.12 ₍₁₂₎	2545.77	0.0508	2337.93 ₍₁₂₎	2381.66	0.3972	1031.69 ₍₂₎	1038.77	1.273	809.01 ₍₈₎	820.58	1.4296	1300.66 ₍₈₎	1411.35	2.4013
3	3136.67 ₍₁₀₎	3327.62	0.0630	2971.46 ₍₉₎	3075.71	0.7898	1085.56 ₍₂₎	1106.50	2.391	869.89 ₍₆₎	879.66	2.701	1456.12 ₍₆₎	1678.23	1.9087
4	6054.28 ₍₉₎	6376.07	0.0799	2973.18 ₍₈₎	3217.73	1.3613	998.48 ₍₁₎	998.48	4.411	1024.31 ₍₅₎	1070.66	5.8528	1478.90 ₍₅₎	1590.34	5.8976
5	3752.04 ₍₁₁₎	3886.42	0.0825	2174.34 ₍₁₁₎	2308.14	0.9214	1014.53 ₍₃₎	1101.46	3.323	740.11 ₍₆₎	747.81	3.8310	1200.45 ₍₇₎	1316.76	4.9598
6	6236.11 ₍₂₃₎	6569.20	0.1228	5970.13 ₍₂₂₎	6191.25	5.0142	1834.74 ₍₂₎	1888.82	11.138	1290.53 ₍₁₃₎	1308.28	17.616	1889.98 ₍₁₄₎	1993.44	20.5612
7	1475.73 ₍₇₎	1583.77	0.0405	1466.58 ₍₈₎	1563.25	0.1554	730.51 ₍₂₎	786.99	0.3428	595.38 ₍₅₎	597.02	0.4937	845.33 ₍₅₎	901.23	0.3565
8	4612.43 ₍₁₂₎	4918.28	0.0837	4310.24 ₍₁₂₎	4535.62	2.4486	1197.32 ₍₁₎	1189.75	5.613	967.84 ₍₈₎	977.43	7.9899	1709.87 ₍₈₎	1876.91	6.4523

Tabla 15: Resumen de los resultados en las ocho instancias TTRP (II).

Problema	MJ			CMT			Kilby			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	603.98 ₍₅₎	616.11	1.8323	822.49 ₍₉₎	881.23	0.3237	1208.06 ₍₁₀₎	1208.06	1.1457	1419.09 ₍₆₎	1535.37	0.2584
2	966.86 ₍₁₀₎	967.12	6.4166	1344.54 ₍₁₅₎	1401.16	1.3781	1712.69 ₍₁₈₎	1712.69	4.9588	2396.65 ₍₁₂₎	2529.72	0.7926
3	1125.96 ₍₈₎	1150.70	14.5486	1267.31 ₍₁₀₎	1334.63	1.8367	1953.44 ₍₁₀₎	2124.76	5.0726	3126.42 ₍₉₎	3323.73	1.8182
4	1216.31 ₍₈₎	1230.55	56.5215	1580.5 ₍₁₀₎	1686.62	2.4692	3385.09 ₍₁₄₎	3385.09	14.4167	5918.16 ₍₉₎	6381.68	3.0313
5	983.85 ₍₁₀₎	1016.63	22.0210	1208.92 ₍₁₃₎	1288.28	1.6385	2682.64 ₍₂₀₎	2682.64	10.1838	3677.54 ₍₁₂₎	3873.64	1.8079
6	1876.13 ₍₁₉₎	1904.92	105.1122	2084.52 ₍₂₂₎	2125.39	13.2150	3858.2 ₍₃₄₎	3858.2	135.5542	6521.21 ₍₂₂₎	6634.31	12.2426
7	631.11 ₍₅₎	645.83	1.8917	813.44 ₍₈₎	891.55	0.2515	1199.14 ₍₁₀₎	1199.14	1.0958	1515.52 ₍₇₎	1601.22	0.2534
8	1264.43 ₍₉₎	1284.69	71.0268	1634.92 ₍₁₆₎	1716.66	4.8927	2733.57 ₍₂₄₎	2733.57	45.2076	4848.46 ₍₁₃₎	4944.19	5.7698

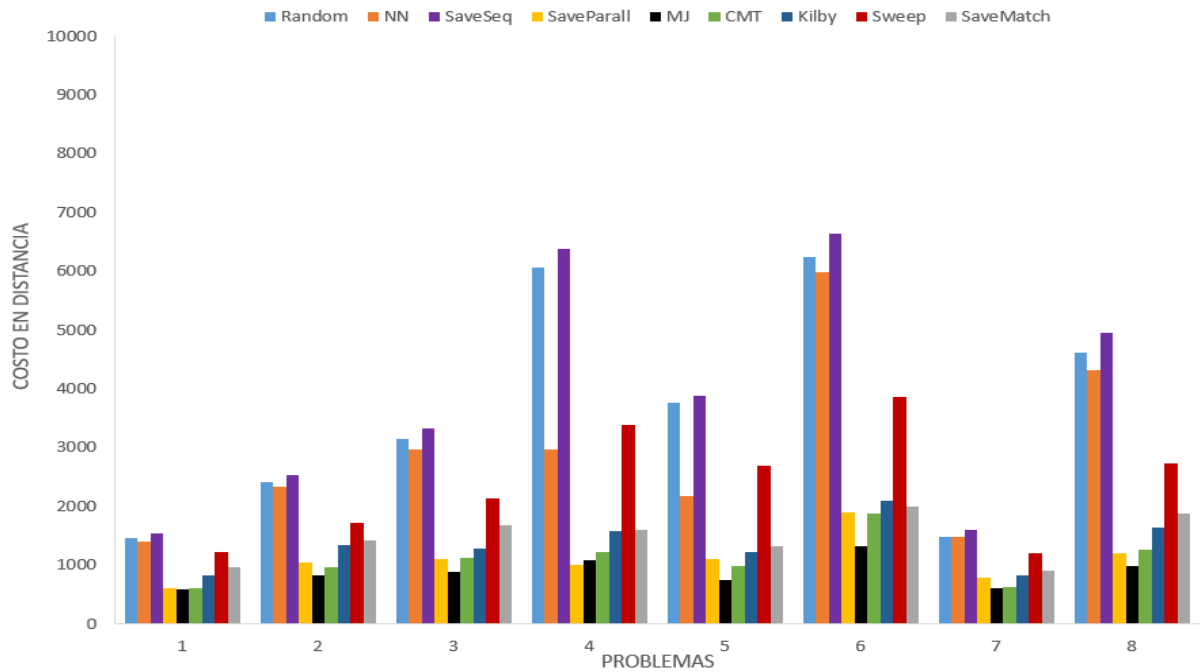


Figura 11: Gráfica de barras para el costo de las heurísticas en la variante TTRP.

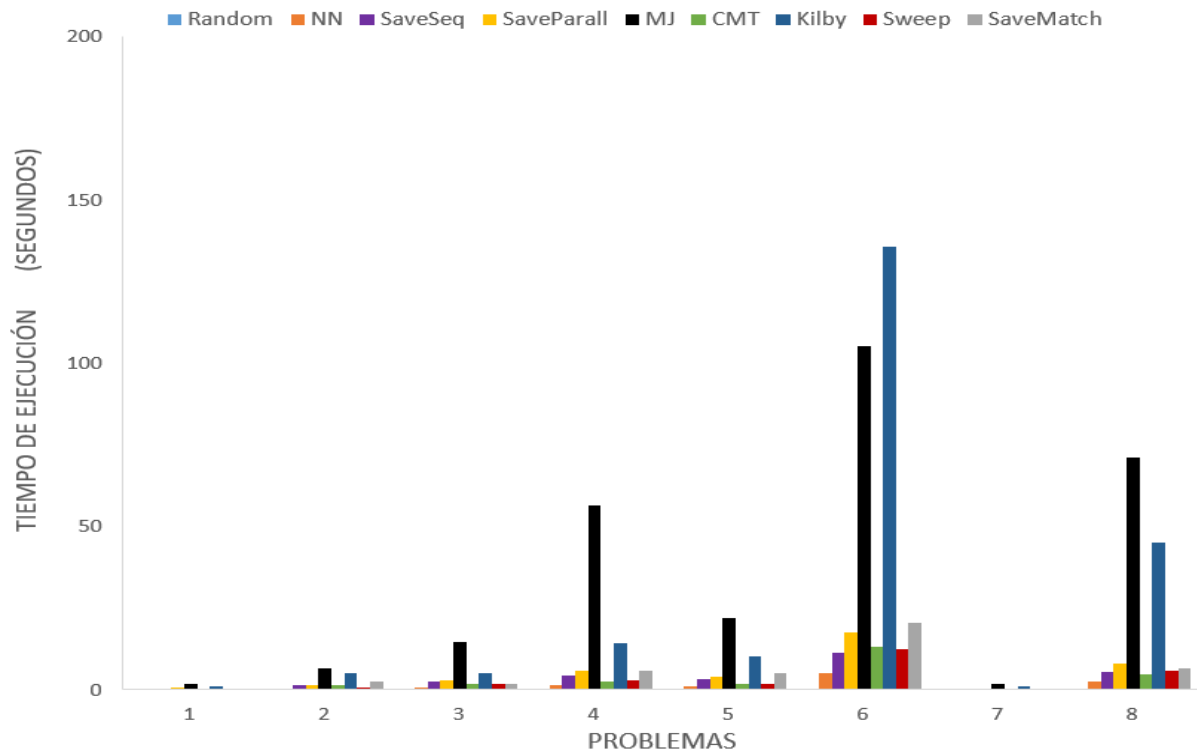


Figura 12: Gráfica de barras para el tiempo de ejecución de las heurísticas en la variante TTRP.

Tabla 16: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias CVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1487.59 ₍₄₎	1702.14	0.034	1092.39 ₍₅₎	1233.61	0.105	567.63 ₍₄₎	575.52	0.087	561.64 ₍₄₎	561.64	0.196
2	2190.94 ₍₆₎	2392.85	0.047	1579.44 ₍₆₎	1749.63	0.087	759.68 ₍₆₎	786.85	0.149	723.2 ₍₆₎	724.73	0.692
3	2126.94 ₍₃₎	2310.81	0.032	630.41 ₍₃₎	632.28	0.123	659.32 ₍₃₎	671.75	0.137	631.85 ₍₃₎	631.85	0.690
4	3082.93 ₍₄₎	3256.04	0.039	1932.01 ₍₄₎	2049.58	0.133	841.33 ₍₄₎	859.02	0.175	762.48 ₍₄₎	762.48	1.644
5	57163.43 ₍₄₎	58923.64	0.049	19443.38 ₍₄₎	20534.06	1.400	6145.42 ₍₄₎	6258.79	1.435	5374.23 ₍₄₎	5374.23	233.309
6	14512.57 ₍₁₈₎	14832.38	0.039	9070.42 ₍₁₉₎	9341.01	0.361	2861.28 ₍₁₈₎	2936.79	0.376	2628.41 ₍₁₉₎	2629.57	22.909
7	41197.21 ₍₃₇₎	42051.26	0.077	27385.16 ₍₃₇₎	27979.17	1.437	7516.1 ₍₃₆₎	7648.00	2.016	6871.46 ₍₃₈₎	6871.46	377.605
8	86706.52 ₍₅₈₎	87593.49	0.088	57024.2 ₍₅₇₎	58727.53	6.434	16998.52 ₍₅₇₎	17235.64	451.87	13678.65 ₍₅₇₎	13678.65	1951.639

Tabla 17: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias CVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	610.01 ₍₄₎	611.59	0.244	741.3 ₍₅₎	854.59	0.114	1298.37 ₍₄₎	1367.20	0.045
2	843.93 ₍₆₎	848.90	0.366	919.7 ₍₇₎	978.99	0.132	1772.65 ₍₆₎	1851.70	0.079
3	659.28 ₍₃₎	664.63	0.482	954.37 ₍₄₎	1004.02	0.201	1728.84 ₍₃₎	1740.60	0.057
4	827.68 ₍₄₎	835.15	0.867	1172.05 ₍₅₎	1286.96	0.168	2134.69 ₍₄₎	2158.63	0.068
5	6181.64 ₍₄₎	6266.93	61.637	16650.47 ₍₅₎	17953.30	1.182	70053.21 ₍₄₎	70053.21	0.157
6	2848.17 ₍₁₈₎	2947.41	1.732	3451.65 ₍₂₁₎	3582.78	0.506	7701.55 ₍₁₉₎	7745.33	0.081
7	7353.73 ₍₃₆₎	7420.82	9.114	9495.97 ₍₄₄₎	9773.22	3.424	22059.61 ₍₃₈₎	22205.85	0.150
8	14919.46 ₍₅₆₎	14948.77	41.346	18770.32 ₍₆₇₎	19176.97	18.213	45755.14 ₍₅₇₎	46104.44	0.361

Tabla 18: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias HFVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1313.9 ₍₃₎	1457.71	0.024	957.99 ₍₃₎	1001.56	0.077	524.66 ₍₃₎	527.10	0.059	536.53 ₍₃₎	536.53	0.270
2	2205.73 ₍₃₎	2275.20	0.025	1273.59 ₍₃₎	1411.39	0.080	645.89 ₍₃₎	652.98	0.093	851.96 ₍₇₎	851.96	1.181
3	2124.18 ₍₃₎	2264.74	0.026	1340.46 ₍₃₎	1429.92	0.106	653.88 ₍₃₎	662.28	0.094	869.89 ₍₆₎	879.66	2.701
4	2983.77 ₍₃₎	3123.60	0.036	1899.58 ₍₃₎	1962.89	0.266	768.28 ₍₃₎	774.38	0.409	1024.31 ₍₅₎	1070.66	5.8528
5	58491.9 ₍₄₎	61785.35	0.037	20348.32 ₍₄₎	22736.22	1.033	6255.43 ₍₄₎	6397.67	1.092	7400.11 ₍₆₎	7470.81	3.8310
6	14529.19 ₍₁₆₎	14755.53	0.091	8904.08 ₍₁₅₎	9131.46	0.547	2534.17 ₍₁₅₎	2588.6	0.673	1290.53 ₍₁₎	1308.28	17.616
7	40180.34 ₍₂₃₎	41402.19	0.123	25685.67 ₍₂₃₎	26688.21	2.158	5511.25 ₍₂₃₎	5597.72	2.527	5955.38 ₍₅₎	5970.02	4.4937
8	85814.92 ₍₅₂₎	87507.12	0.070	56058.93 ₍₅₂₎	57878.02	7.989	15294.68 ₍₅₁₎	15672.3	20.9242	16967.84 ₍₈₎	977.43	7.9899

Tabla 19: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias HFVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	536.15 ₍₃₎	538.77	0.186	652.83 ₍₃₎	690.15	0.076	1280.30 ₍₃₎	1280.31	0.019
2	681.73 ₍₃₎	686.02	0.358	911.82 ₍₄₎	972.90	0.102	1720.60 ₍₃₎	1723.95	0.059
3	662.41 ₍₃₎	665.11	0.372	922.66 ₍₃₎	993.36	0.098	1725.50 ₍₃₎	1731.74	0.037
4	782.9 ₍₃₎	784.41	4.810	1180.74 ₍₃₎	1242.09	0.867	2122.11 ₍₃₎	2123.70	0.087
5	6453.51 ₍₄₎	6537.06	51.711	16345.57 ₍₅₎	20324.56	1.245	70056.72 ₍₄₎	70279.88	0.138
6	2711.9 ₍₁₅₎	2714.86	2.892	3189.95 ₍₁₇₎	3454.43	0.698	7890.30 ₍₁₅₎	7904.14	0.221
7	5426.43 ₍₂₃₎	5545.46	15.457	7739.26 ₍₂₅₎	8646.55	2.866	20987.24 ₍₂₄₎	21087.88	0.271
8	13949.38 ₍₅₁₎	13952.11	50.595	20424.46 ₍₅₄₎	22242.66	17.180	45301.08 ₍₅₂₎	45417.50	0.448

Tabla 20: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias MDVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	900.42 ₍₁₂₎	957.80	0.032	802.75 ₍₁₂₎	826.44	0.037	636.64 ₍₁₂₎	646.41	0.048	626.2 ₍₁₃₎	626.2	0.058
2	2361.66 ₍₉₎	2563.63	0.040	1373.25 ₍₉₎	1501.49	0.082	878.92 ₍₉₎	901.40	0.115	833.09 ₍₉₎	833.09	0.324
3	7956.15 ₍₁₆₎	8193.67	0.043	4820.68 ₍₁₆₎	5110.09	0.132	3273.09 ₍₁₆₎	3312.51	0.133	3302.04 ₍₁₆₎	3302.04	0.280
4	18286.31 ₍₃₆₎	18825.05	0.064	11459.03 ₍₃₇₎	11864.58	0.211	7456.18 ₍₃₆₎	7543.33	0.262	7429.57 ₍₃₆₎	7429.57	0.583
5	10140.46 ₍₂₃₎	10529.67	0.065	6203.79 ₍₂₃₎	6387.67	0.189	3344.44 ₍₂₂₎	3437.04	0.335	3016.17 ₍₂₃₎	3016.17	1.888
6	13518.87 ₍₂₇₎	14134.88	0.062	7890.4 ₍₂₇₎	8066.29	0.160	4586.49 ₍₂₇₎	4727.56	0.261	3993.25 ₍₂₇₎	3993.25	1.241
7	856.98 ₍₇₎	902.52	0.029	704.23 ₍₇₎	740.36	0.032	525.66 ₍₇₎	529.71	0.048	528.52 ₍₇₎	528.52	0.059
8	5309.65 ₍₁₁₎	5609.60	0.040	3408.17 ₍₁₁₎	3621.49	0.081	2078.62 ₍₁₁₎	2123.68	0.115	1951.45 ₍₁₂₎	1951.45	0.226

Tabla 21: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias MDVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	639.01 ₍₁₂₎	639.01	0.062	733.42 ₍₁₄₎	759.23	0.044	823.08 ₍₁₂₎	850.86	0.030
2	884.87 ₍₉₎	894.12	0.216	1114.1 ₍₁₁₎	1155.09	0.106	1660.64 ₍₉₎	1691.91	0.052
3	3237.39 ₍₁₆₎	3237.39	0.303	3943.05 ₍₂₁₎	4288.36	0.141	6067.77 ₍₁₆₎	6106.68	0.059
4	7284.11 ₍₃₆₎	7284.11	0.995	9646.23 ₍₄₅₎	10024.89	0.308	12779.03 ₍₃₆₎	12862.69	0.077
5	3277.74 ₍₂₃₎	3287.15	1.320	4421.83 ₍₂₆₎	4602.53	0.374	7486.01 ₍₂₃₎	7632.53	0.078
6	4341.31 ₍₂₇₎	4343.88	0.701	5489.8 ₍₃₀₎	5815.21	0.240	9081.2 ₍₂₈₎	9156.59	0.101
7	526.21 ₍₇₎	526.21	0.065	611.04 ₍₇₎	643.19	0.057	756.73 ₍₇₎	773.11	0.033
8	2041.57 ₍₁₁₎	2089.05	0.348	2744.63 ₍₁₃₎	2879.68	0.143	4042.76 ₍₁₁₎	4110.23	0.051

Tabla 22: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias TTRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1553.98 ₍₆₎	1627.30	0.052	1167.8 ₍₅₎	1259.35	0.078	602.8 ₍₁₎	602.8	0.117	765.25 ₍₅₎	765.25	0.196
2	2528.93 ₍₁₃₎	2672.96	0.127	1798.35 ₍₁₁₎	1877.47	0.256	1020.68 ₍₂₎	1033.49	0.273	1809.01 ₍₈₎	1820.58	1.4296
3	3229.75 ₍₉₎	3423.11	0.051	2226.32 ₍₈₎	2321.50	0.231	1085.56 ₍₂₎	1106.50	0.391	1869.89 ₍₆₎	1879.66	2.701
4	6122.35 ₍₇₎	6371.67	0.072	2059.33 ₍₈₎	2152.60	0.368	998.48 ₍₁₎	998.48	0.411	1024.31 ₍₅₎	1070.66	5.8528
5	3831.82 ₍₁₁₎	4085.60	0.086	1382.79 ₍₉₎	1733.76	0.316	1012.99 ₍₃₎	1101.46	0.323	1740.11 ₍₆₎	1747.81	3.8310
6	6491.96 ₍₂₂₎	6679.67	0.066	4416.71 ₍₂₀₎	4553.98	1.191	1834.74 ₍₂₎	1888.82	1.138	1990.53 ₍₁₃₎	2308.28	17.616
7	1519.75 ₍₆₎	1632.47	0.040	1158.41 ₍₆₎	1242.58	0.104	729.48 ₍₂₎	782.74	0.128	1595.38 ₍₅₎	1597.02	0.4937
8	5020.74 ₍₁₀₎	5140.10	0.056	3299.86 ₍₁₃₎	3440.13	0.618	1157.44 ₍₁₎	1166.62	0.613	1967.84 ₍₈₎	1977.43	7.9899

Tabla 23: Resumen de los resultados obtenidos por BHCVRP (Díaz-Subi, 2016) en las ocho instancias TTRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	793.95 ₍₅₎	793.95	0.236	860.06 ₍₉₎	934.13	0.217	1419.09 ₍₆₎	1535.37	0.2584
2	1265.01 ₍₁₀₎	1269.96	0.455	1352.59 ₍₁₅₎	1417.89	0.421	2396.65 ₍₁₂₎	2529.72	0.7926
3	1180.2 ₍₉₎	1185.81	1.049	1336.94 ₍₁₁₎	1423.57	0.326	3126.42 ₍₉₎	3323.73	1.8182
4	1646.93 ₍₈₎	1646.97	2.480	1761.11 ₍₁₁₎	1835.43	0.524	5918.16 ₍₉₎	6381.68	3.0313
5	1202.0 ₍₁₀₎	1202.0	1.071	1340.68 ₍₁₃₎	1442.07	0.439	3677.54 ₍₁₂₎	3873.64	1.8079
6	1942.7 ₍₁₉₎	1945.89	5.621	2278.04 ₍₂₅₎	2353.39	1.770	6521.21 ₍₂₂₎	6634.31	12.2426
7	871.83 ₍₅₎	874.40	0.255	843.97 ₍₉₎	917.22	0.150	1515.52 ₍₇₎	1601.22	0.2534
8	1578.05 ₍₁₁₎	1603.21	3.825	1883.48 ₍₁₆₎	1986.04	0.741	4848.46 ₍₁₃₎	4944.19	5.7698

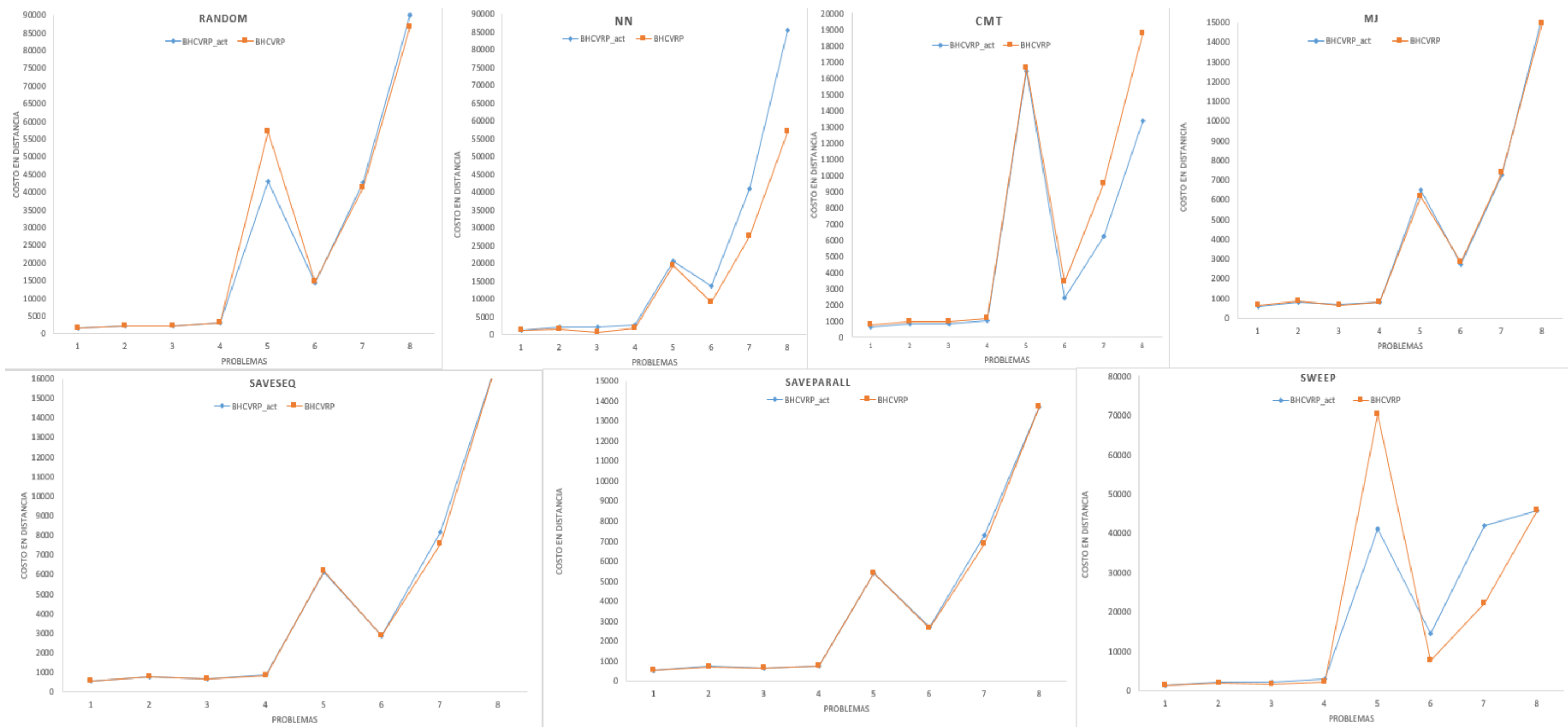


Figura 13: Comparación del comportamiento de las heurísticas de BHCVRP en ambas versiones en cuanto al costo en distancia de las soluciones para la variante CVRP.

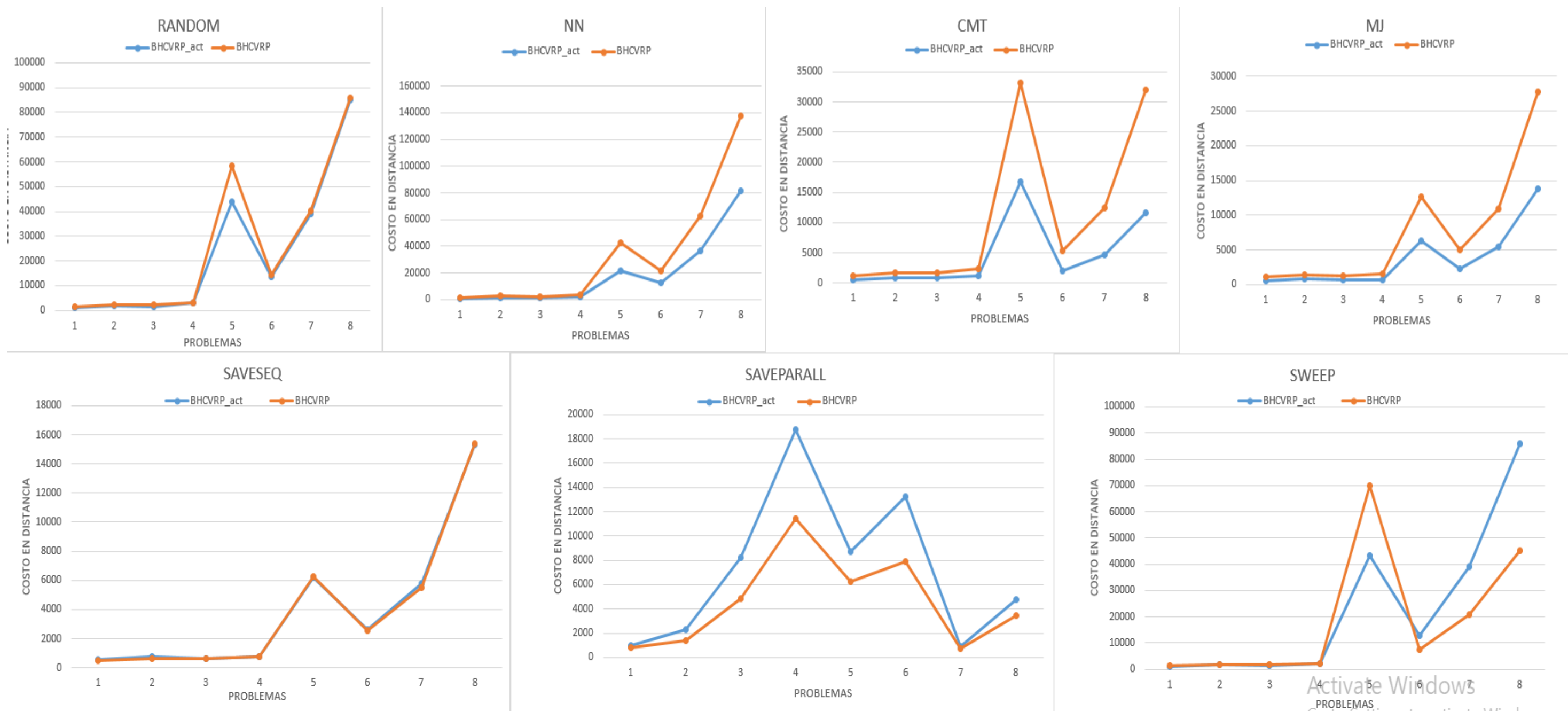


Figura 14: Comparación del comportamiento de las heurísticas de BHCVRP en ambas versiones en cuanto al costo en distancia de las soluciones para la variante HFVRP.

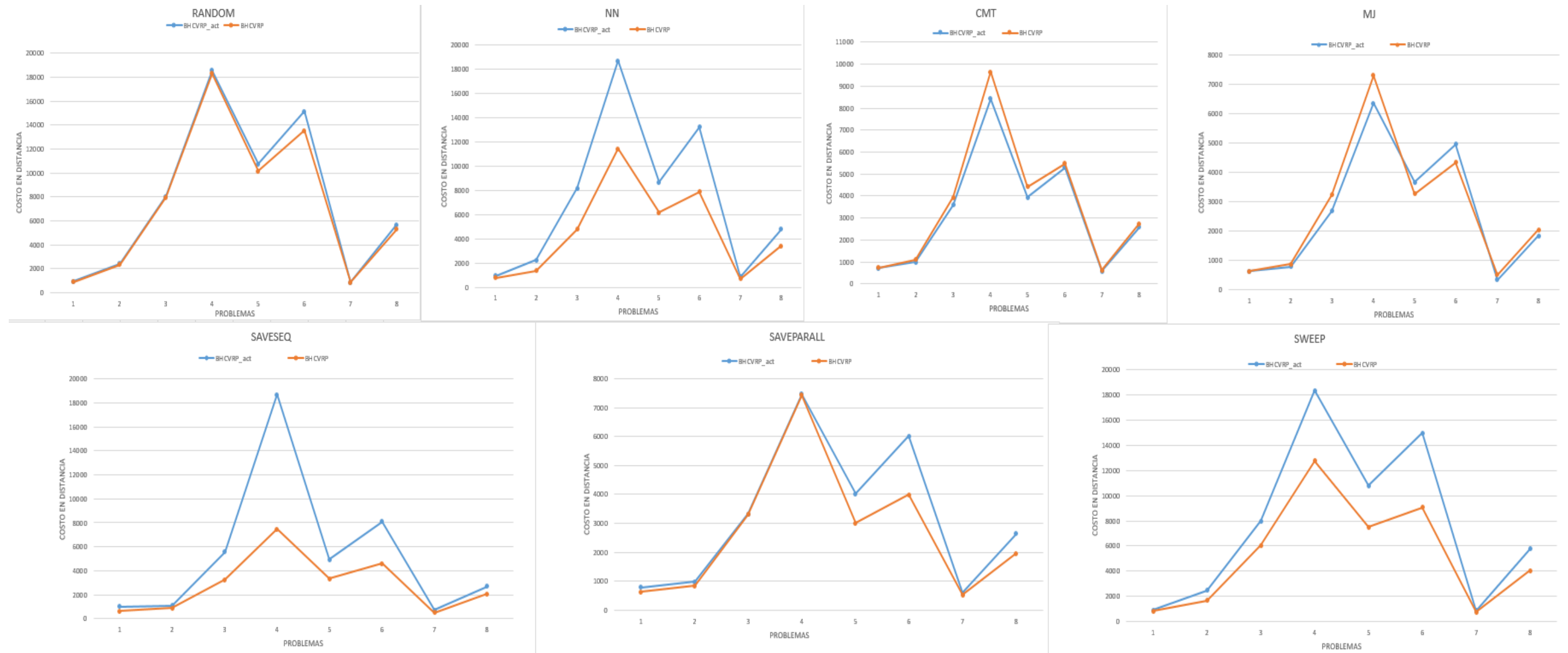


Figura 15: Comparación del comportamiento de las heurísticas de BHCVRP en ambas versiones en cuanto al costo en distancia de las soluciones para la variante MDVRP.

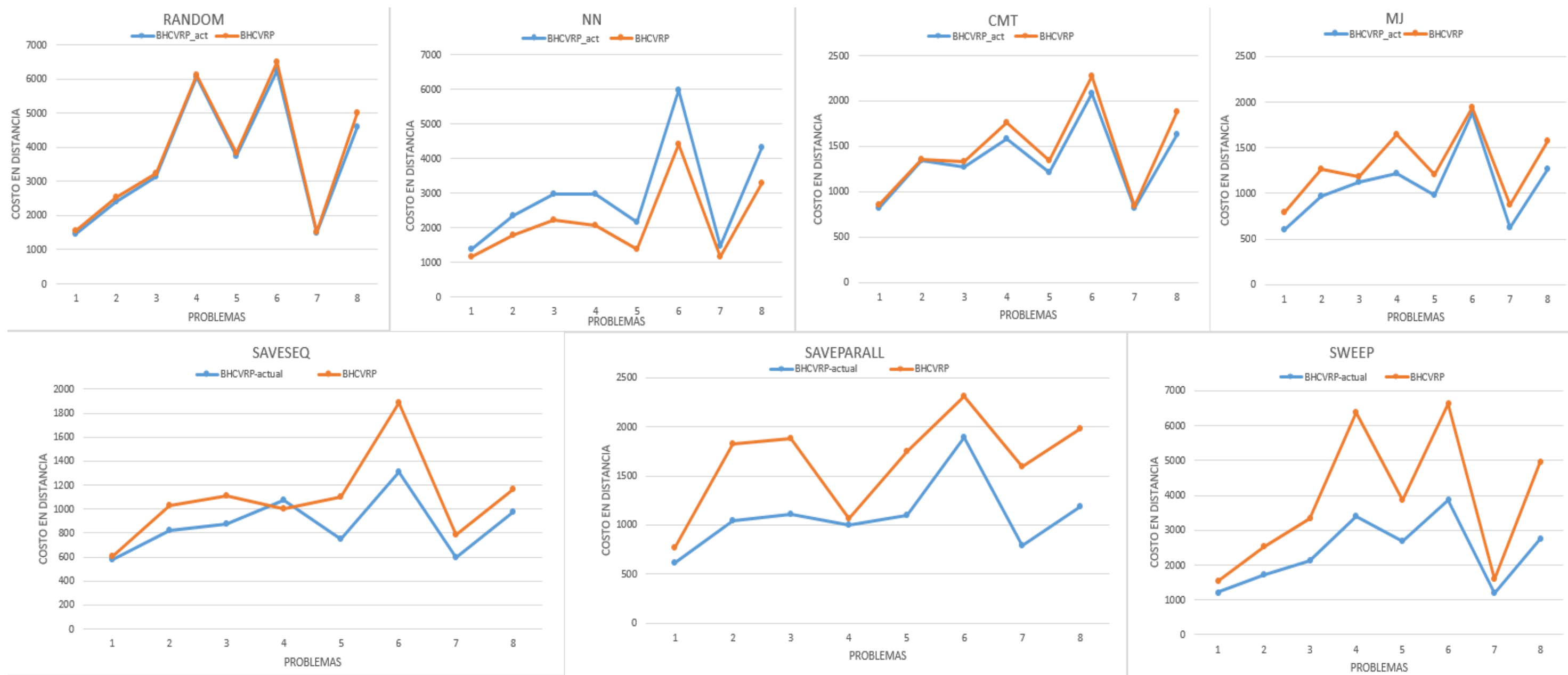


Figura 16: Comparación del comportamiento de las heurísticas de BHCVRP en ambas versiones en cuanto al costo en distancia de las soluciones para la variante TTRP.

Tabla 24: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a costo para la variante CVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	14.0	14.0	≥ 0.2	0.932647
<i>Random</i> vs <i>Random_act</i>	14.0	14.0	≥ 0.2	0.932647
<i>Sweep_act</i> vs <i>Sweep</i>	7.0	21.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	21.0	7.0	≥ 0.2	0.204894
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	3.0	25.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	25.0	3.0	≥ 0.2	0.051913
<i>SaveParall_act</i> vs <i>SaveParall</i>	4.0	24.0	≥ 0.2	1
<i>SaveParall</i> vs <i>SaveParall_act</i>	24.0	4.0	≥ 0.2	0.075927
<i>MJ_act</i> vs <i>MJ</i>	20.0	16.0	≥ 0.2	0.726286
<i>MJ</i> vs <i>MJ_act</i>	16.0	20.0	≥ 0.2	1
<i>NN_act</i> vs <i>NN</i>	0.0	36.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act</i> vs <i>CMT</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT</i> vs <i>CMT_act</i>	0.0	36.0	≥ 0.2	1

Tabla 25: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a costo para la variante HFVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	28.0	0.0	≥ 0.2	0.014248
<i>Random</i> vs <i>Random_act</i>	0.0	28.0	≥ 0.2	1
<i>Sweep_act</i> vs <i>Sweep</i>	11.0	17.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	17.0	11.0	≥ 0.2	0.554113
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	9.0	19.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	19.0	9.0	≥ 0.2	0.352542
<i>SaveParall_act</i> vs <i>SaveParall</i>	14.0	14.0	≥ 0.2	0.932647
<i>SaveParall</i> vs <i>SaveParall_act</i>	14.0	14.0	≥ 0.2	0.932647
<i>MJ_act</i> vs <i>MJ</i>	18.0	10.0	≥ 0.2	0.446873
<i>MJ</i> vs <i>MJ_act</i>	10.0	18.0	≥ 0.2	1
<i>NN_act</i> vs <i>NN</i>	3.0	25.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	25.0	3.0	≥ 0.2	0.051913
<i>CMT_act</i> vs <i>CMT</i>	24.0	4.0	≥ 0.2	0.075927
<i>CMT</i> vs <i>CMT_act</i>	4.0	24.0	≥ 0.2	1

Tabla 26: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a costo para la variante MDVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act vs Random</i>	1.0	35.0	≥ 0.2	1
<i>Random vs Random_act</i>	35.0	1.0	≥ 0.2	0.014266
<i>Sweep_act vs Sweep</i>	0.0	36.0	≥ 0.2	1
<i>Sweep vs Sweep_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveSeq_act vs SaveSeq</i>	0.0	36.0	≥ 0.2	1
<i>SaveSeq vs SaveSeq_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveParall_act vs SaveParall</i>	0.0	36.0	≥ 0.2	1
<i>SaveParall vs SaveParall_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>MJ_act vs MJ</i>	23.0	13.0	≥ 0.2	0.441209
<i>MJ vs MJ_act</i>	13.0	23.0	≥ 0.2	1
<i>NN_act vs NN</i>	0.0	36.0	≥ 0.2	1
<i>NN vs NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act vs CMT</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT vs CMT_act</i>	0.0	36.0	≥ 0.2	1

Tabla 27: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a costo para la variante TTRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act vs Random</i>	36.0	0.0	≥ 0.2	0.009583
<i>Random vs Random_act</i>	0.0	36.0	≥ 0.2	1
<i>Sweep_act vs Sweep</i>	23.0	13.0	≥ 0.2	0.441209
<i>Sweep vs Sweep_act</i>	13.0	23.0	≥ 0.2	1
<i>SaveSeq_act vs SaveSeq</i>	11.0	17.0	≥ 0.2	1
<i>SaveSeq vs SaveSeq_act</i>	17.0	11.0	≥ 0.2	0.554113
<i>SaveParall_act vs SaveParall</i>	9.0	19.0	≥ 0.2	1
<i>SaveParall vs SaveParall_act</i>	19.0	9.0	≥ 0.2	0.352542
<i>MJ_act vs MJ</i>	36.0	0.0	≥ 0.2	0.009583
<i>MJ vs MJ_act</i>	0.0	36.0	≥ 0.2	1
<i>NN_act vs NN</i>	0.0	36.0	≥ 0.2	1
<i>NN vs NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act vs CMT</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT vs CMT_act</i>	0.0	36.0	≥ 0.2	1

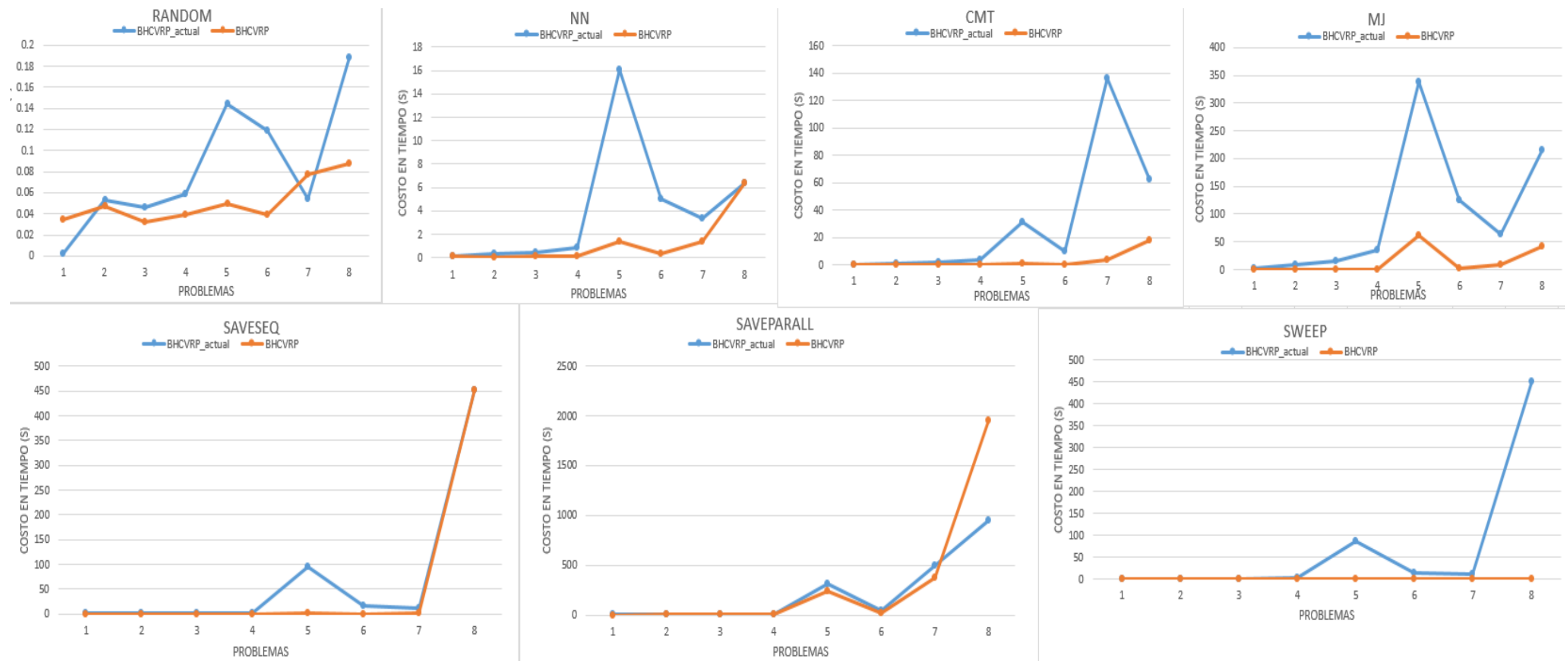


Figura 17: Comparación del comportamiento de las heurísticas de BHCVRP en ambas versiones en cuanto al costo en tiempo de las soluciones para la variante CVRP.

Es importante aclarar, que para el resto de las variantes VRP el comportamiento de las heurísticas para ambas versiones de BCHVRP en cuanto a costo en tiempo de ejecución de las soluciones es similar.

Tabla 28: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a tiempo para la variante CVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	9.0	27.0	≥ 0.2	1
<i>Random</i> vs <i>Random_act</i>	27.0	9.0	≥ 0.2	0.183431
<i>Sweep_act</i> vs <i>Sweep</i>	0.0	36.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	0.0	28.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	28.0	0.0	≥ 0.2	0.014248
<i>SaveParall_act</i> vs <i>SaveParall</i>	8.0	28.0	≥ 0.2	1
<i>SaveParall</i> vs <i>SaveParall_act</i>	28.0	8.0	≥ 0.2	0.141482
<i>MJ_act</i> vs <i>MJ</i>	0.0	36.0	≥ 0.2	1
<i>MJ</i> vs <i>MJ_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>NN_act</i> vs <i>NN</i>	0.0	36.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act</i> vs <i>CMT</i>	0.0	36.0	≥ 0.2	1
<i>CMT</i> vs <i>CMT_act</i>	36.0	0.0	≥ 0.2	0.009583

Tabla 29: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a tiempo para la variante HFVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	0.0	36.0	≥ 0.2	1
<i>Random</i> vs <i>Random_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>Sweep_act</i> vs <i>Sweep</i>	0.0	36.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	0.0	28.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	28.0	0.0	≥ 0.2	0.014248
<i>SaveParall_act</i> vs <i>SaveParall</i>	0.0	36.0	≥ 0.2	1
<i>SaveParall</i> vs <i>SaveParall_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>MJ_act</i> vs <i>MJ</i>	0.0	36.0	≥ 0.2	1
<i>MJ</i> vs <i>MJ_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>NN_act</i> vs <i>NN</i>	0.0	36.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act</i> vs <i>CMT</i>	0.0	36.0	≥ 0.2	1
<i>CMT</i> vs <i>CMT_act</i>	36.0	0.0	≥ 0.2	0.009583

Tabla 30: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a tiempo para la variante MDVRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	0.0	36.0	≥ 0.2	1
<i>Random</i> vs <i>Random_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>Sweep_act</i> vs <i>Sweep</i>	0.0	36.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	0.0	36.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveParall_act</i> vs <i>SaveParall</i>	0.0	36.0	≥ 0.2	1
<i>SaveParall</i> vs <i>SaveParall_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>MJ_act</i> vs <i>MJ</i>	0.0	36.0	≥ 0.2	1
<i>MJ</i> vs <i>MJ_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>NN_act</i> vs <i>NN</i>	0.0	36.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act</i> vs <i>CMT</i>	0.0	36.0	≥ 0.2	1
<i>CMT</i> vs <i>CMT_act</i>	36.0	0.0	≥ 0.2	0.009583

Tabla 31: Resultados de la prueba de Wilcoxon en la comparación con los algoritmos de BHCVRP (Díaz-Subi, 2016) en cuanto a tiempo para la variante TTRP.

Hipótesis	R+	R-	<i>p-value</i>	<i>p-value</i> asintótico
<i>Random_act</i> vs <i>Random</i>	0.0	36.0	≥ 0.2	1
<i>Random</i> vs <i>Random_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>Sweep_act</i> vs <i>Sweep</i>	0.0	36.0	≥ 0.2	1
<i>Sweep</i> vs <i>Sweep_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveSeq_act</i> vs <i>SaveSeq</i>	0.0	36.0	≥ 0.2	1
<i>SaveSeq</i> vs <i>SaveSeq_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>SaveParall_act</i> vs <i>SaveParall</i>	0.0	36.0	≥ 0.2	1
<i>SaveParall</i> vs <i>SaveParall_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>MJ_act</i> vs <i>MJ</i>	0.0	36.0	≥ 0.2	1
<i>MJ</i> vs <i>MJ_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>NN_act</i> vs <i>NN</i>	0.0	36.0	≥ 0.2	1
<i>NN</i> vs <i>NN_act</i>	36.0	0.0	≥ 0.2	0.009583
<i>CMT_act</i> vs <i>CMT</i>	0.0	36.0	≥ 0.2	1
<i>CMT</i> vs <i>CMT_act</i>	36.0	0.0	≥ 0.2	0.009583

Tabla 32: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias CVRP.

Problema	BHCVRP			OR-Tools		
	Costo en distancia	Total de rutas	Tiempo de ejecución (s)	Costo en distancia	Total de rutas	Tiempo de ejecución (s)
1	558.89	4	0.54	2152.00	4	0.13
2	787.61	6	1.25	3916.00	6	0.33
3	666.86	3	15.75	4133.00	3	0.27
4	814.49	4	36.10	4534.00	4	0.57
5	6134.72	4	94.80	9298.00	4	15.00
6	2391.66	18	9.71	2597.00	19	1.61
7	6239.59	36	136.57	6970.00	37	9.12
8	13386.91	56	62.29	13881.00	59	15.00

Tabla 33: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias HFVRP.

Problema	BHCVRP			OR-Tools		
	Costo en distancia	Total de rutas	Tiempo de ejecución (s)	Costo en distancia	Total de rutas	Tiempo de ejecución (s)
1	550.34	4	0.48	3225.00	4	0.14
2	799.87	6	9.36	3730.00	4	0.30
3	666.31	3	1.35	4030.00	3	0.40
4	768.28	3	2.66	1512.00	3	0.71
5	6125.32	4	24.61	9327.00	4	12.03
6	2118.76	15	19.99	2597.00	19	1.61
7	4705.29	23	121.95	6970.00	37	9.12
8	11564.10	51	1003.68	13881.00	59	15.00

Tabla 34: Resultados de la prueba de Wilcoxon en la comparación con los resultados de BHCVRP y OR-Tools en cuanto a costo para las variantes CVRP y HFVRP.

Hipótesis	R+	R-	<i>pvalue</i>	<i>p-value</i> asintótico
<u>Variante CVRP</u>				
<i>BHCVRP vs OR-Tools</i>	36.0	0.0	≥ 0.2	0.009583
<i>OR-Tools vs BHCVRP</i>	0.0	36.0	≥ 0.2	1
<u>Variante HFVRP</u>				
<i>BHCVRP vs OR-Tools</i>	36.0	0.0	≥ 0.2	0.009583
<i>OR-Tools vs BHCVRP</i>	0.0	36.0	≥ 0.2	1

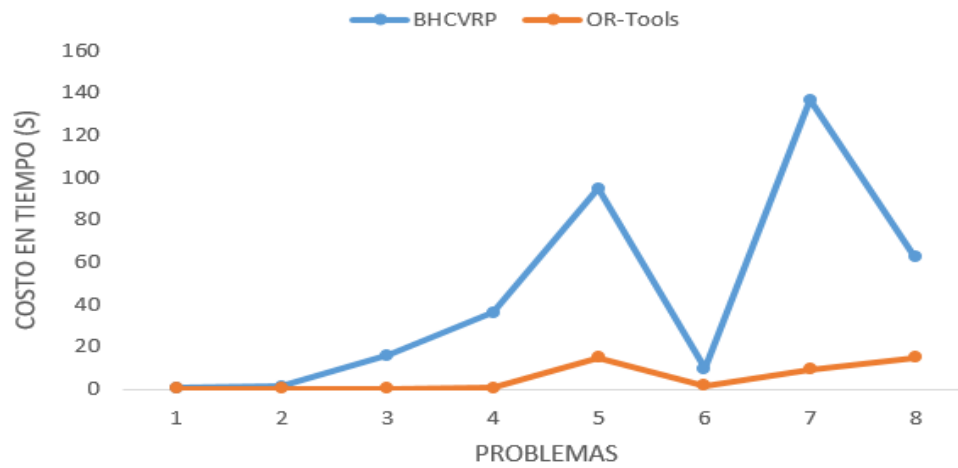


Figura 18: Comportamiento de las soluciones obtenidas por BHCVRP y OR-Tools en cuanto a tiempo de ejecución para la variante CVRP.

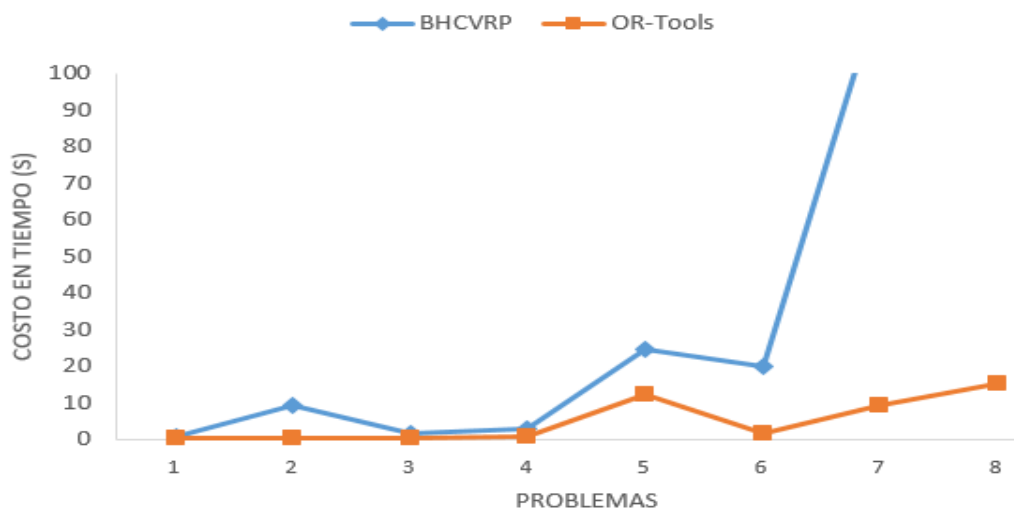


Figura 19: Comportamiento de las soluciones obtenidas por BHCVRP y OR-Tools en cuanto a tiempo de ejecución para la variante HFVRP.

Tabla 35: Resultados de la prueba de Wilcoxon en la comparación con los resultados de BHCVRP y OR-Tools en cuanto a tiempo de ejecución para las variantes CVRP y HFVRP.

Hipótesis	R+	R-	<i>pvalue</i>	<i>p-value</i> asintótico
<u>Variante CVRP</u>				
<i>BHCVRP vs OR-Tools</i>	0.0	36.0	≥ 0.2	1
<i>OR-Tools vs BHCVRP</i>	36.0	0.0	≥ 0.2	0.009583
<u>Variante HFVRP</u>				
<i>BHCVRP vs OR-Tools</i>	0.0	36.0	≥ 0.2	1
<i>OR-Tools vs BHCVRP</i>	36.0	0.0	≥ 0.2	0.009583